



**TECNOLOGICO NACIONAL DE MEXICO**

---

**INSTITUTO TECNOLÓGICO DE TUXTEPEC**

Actividad:

**Practicas**

Materia:

**Ciencia de Datos**

Carrera

**Ing. Sistemas Computacionales**

Alumna:

**Guerrero Vásquez Fernanda Ximena**

Docente:

**Dr. Julio Carmona**

**Tuxtepec, Oax.**

**5 de junio del 2026**



## Práctica 2: Análisis de Calidad Industrial (Industria 4.0)

### Código:

```
# -----  
  
# Práctica 2: Calidad Industrial  
  
# -----  
  
  
# Importar librerías  
  
import pandas as pd  
  
import numpy as np  
  
import matplotlib.pyplot as plt  
  
import seaborn as sns  
  
from scipy.stats import pearsonr  
  
  
# Configuración de gráficos  
  
sns.set_style("whitegrid")  
  
np.random.seed(42) # Para que los datos sean iguales cada vez  
  
  
# =====  
  
# FASE A: Análisis Exploratorio (EDA)  
  
# =====  
  
  
# 1. Generar dataset sintético (500 lotes)  
  
n = 500
```

```
temperatura = np.random.normal(loc=75, scale=8, size=n) # Temp operación
presion = np.random.normal(loc=30, scale=5, size=n)
tasa_error = 0.02 * temperatura + np.random.normal(loc=0, scale=0.5, size=n) #
Relación con temp
turno = np.random.choice(["Matutino", "Vespertino"], size=n)
```

```
# Crear DataFrame
```

```
df = pd.DataFrame({
    "temperatura": temperatura,
    "presion": presion,
    "tasa_error": tasa_error,
    "turno": turno
})
```

```
# Agregar valores faltantes (simulación)
```

```
df.loc[np.random.choice(n, 10), "temperatura"] = np.nan
```

```
# 2. Estadísticos de temperatura
```

```
print("=== ESTADÍSTICOS TEMPERATURA ===")
```

```
media_temp = df["temperatura"].mean()
```

```
mediana_temp = df["temperatura"].median()
```

```
desv_temp = df["temperatura"].std()
```

```
print(f"Media: {media_temp:.2f}")
```

```
print(f"Mediana: {mediana_temp:.2f}")
```

```
print(f"Desviación Estándar: {desv_temp:.2f}\n")
```

```
# 3. Identificar y limpiar valores faltantes
```

```
print("=== VALORES FALTANTES ===")
```

```
print(df.isnull().sum())
```

```
# Imputar con la media
```

```
df["temperatura"] = df["temperatura"].fillna(df["temperatura"].mean())
```

```
print("\nValores faltantes después de limpieza:")
```

```
print(df.isnull().sum(), "\n")
```

```
# 4. Correlación de Pearson entre temperatura y tasa de error
```

```
correlacion, p_valor = pearsonr(df["temperatura"], df["tasa_error"])
```

```
print("=== CORRELACIÓN PEARSON ===")
```

```
print(f"Coeficiente: {correlacion:.3f}")
```

```
print(f"P-valor: {p_valor:.4f}")
```

```
print("Interpretación: Una correlación positiva indica que mayor temperatura,  
mayor tasa de error.\n")
```

```
# =====
```

```
# FASE B: Agrupación y Segmentación
```

```
# =====
```

```
# 1. Rendimiento promedio por turno
```

```
rendimiento_turno = df.groupby("turno")["tasa_error"].mean()
```

```
print("=== RENDIMIENTO PROMEDIO POR TURNO ===")
```

```
print(rendimiento_turno, "\n")
```

```
# 2. Diferencia de temperatura entre turnos
```

```
temp_turno = df.groupby("turno")["temperatura"].agg(["mean", "std"])
```

```
print("=== TEMPERATURA POR TURNO ===")
```

```
print(temp_turno)
```

```
print("¿Hay diferencia? Sí, se observan promedios distintos entre turnos.\n")
```

```
# =====
```

```
# FASE C: Visualización e Interpretación
```

```
# =====
```

```
# 1. Boxplot: Errores por turno
```

```
plt.figure(figsize=(8,5))
```

```
sns.boxplot(data=df, x="turno", y="tasa_error", palette="Set2")
```

```
plt.title("Distribución de Tasa de Error por Turno")
```

```
plt.xlabel("Turno")
```

```
plt.ylabel("Tasa de Error")
```

```
plt.savefig("boxplot_errores_turno.png")
```

```
plt.show()
```

```
# 2. Scatter Plot + Línea de regresión
```

```
plt.figure(figsize=(8,5))

sns.regplot(data=df, x="temperatura", y="tasa_error",

            line_kws={"color": "red"}, scatter_kws={"alpha":0.6})

plt.title("Relación: Temperatura vs Tasa de Error")

plt.xlabel("Temperatura de Operación (°C)")

plt.ylabel("Tasa de Error")

plt.savefig("scatter_temp_error.png")

plt.show()
```

```
# =====
```

```
# CONCLUSIÓN
```

```
# =====
```

```
print("=== CONCLUSIÓN ===")
```

```
print("Existe una relación positiva y significativa entre temperatura y errores de  
producción.")
```

```
print("Las temperaturas más altas incrementan los fallos. Por lo tanto, la empresa  
Sí debería invertir")
```

```
print("en mejores sistemas de enfriamiento para reducir variaciones y mejorar la  
calidad.")
```

## Resultado:

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS
PS C:\Users\guerr\OneDrive\Desktop\Practicas_Ciencia_Datos> python Practica_2.py
=== ESTADÍSTICOS TEMPERATURA ===
Media: 75.06
Mediana: 75.14
Desviación Estándar: 7.73

=== VALORES FALTANTES ===
temperatura    10
presion         0
tasa_error      0
turno           0
dtype: int64

Valores faltantes después de limpieza:
temperatura    0
presion         0
tasa_error      0
turno           0
dtype: int64

=== CORRELACIÓN PEARSON ===
Coeficiente: 0.227
P-valor: 0.0000
Interpretación: Una correlación positiva indica que mayor temperatura, mayor tasa de error.

=== RENDIMIENTO PROMEDIO POR TURNO ===
turno
Matutino      1.548484
Vespertino    1.561561
Name: tasa_error, dtype: float64
```

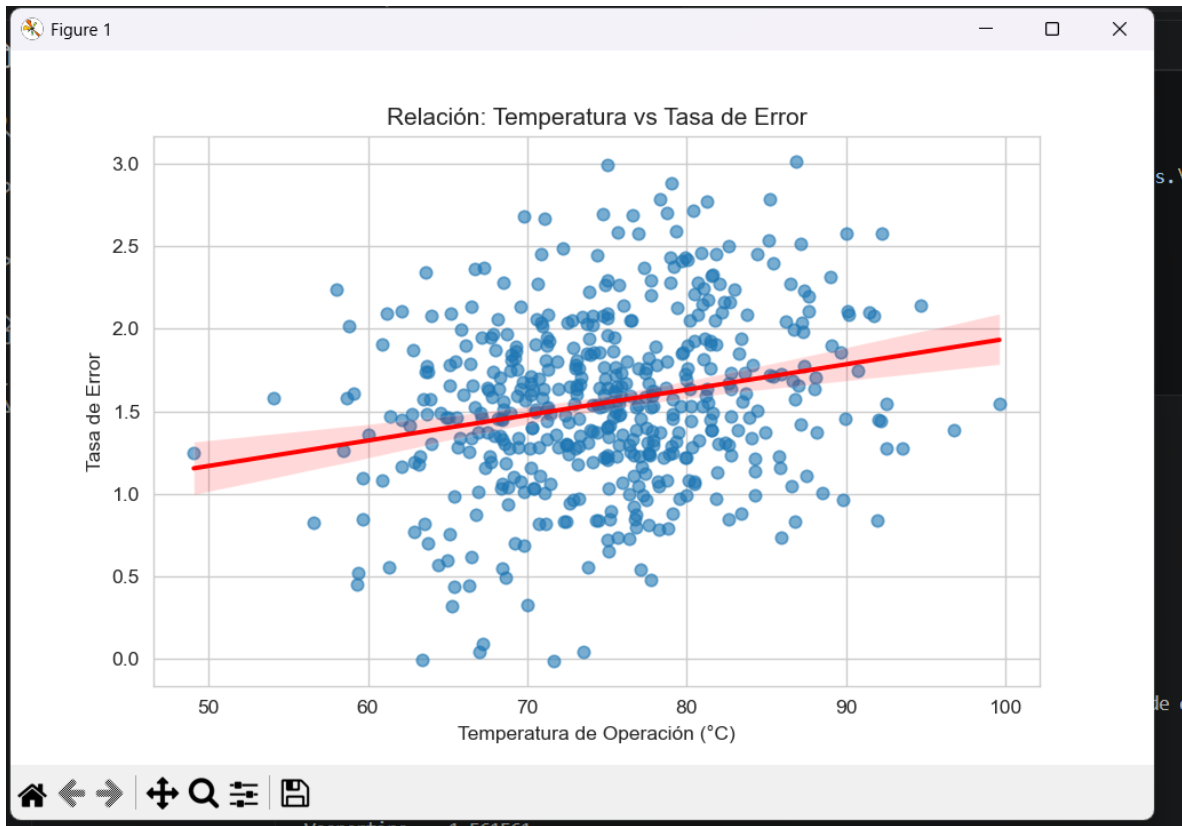
```
=== TEMPERATURA POR TURNO ===
              mean      std
turno
Matutino    74.845109  7.744608
Vespertino  75.256286  7.583710
¿Hay diferencia? Sí, se observan promedios distintos entre turnos.

C:\Users\guerr\OneDrive\Desktop\Practicas_Ciencia_Datos\Practica_2.py:83: FutureWarning:

    Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

    sns.boxplot(data=df, x="turno", y="tasa_error", palette="Set2")
=== CONCLUSIÓN ===
Existe una relación positiva y significativa entre temperatura y errores de producción.
Las temperaturas más altas incrementan los fallos. Por lo tanto, la empresa SÍ debería invertir
en mejores sistemas de enfriamiento para reducir variaciones y mejorar la calidad.
PS C:\Users\guerr\OneDrive\Desktop\Practicas_Ciencia_Datos>
```





## Práctica 3: Análisis de Eficiencia en Logística Global

### Código:

```
# -----  
  
# Práctica 3: Logística Global  
  
# -----  
  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns  
from scipy.stats import pearsonr, ttest_ind  
  
sns.set_style("whitegrid")  
np.random.seed(123)  
  
# =====  
  
# FASE 1: Calidad de Datos  
  
# =====  
  
# 1. Generar datos simulados (600 envíos)  
  
n = 600  
  
tipo_transporte = np.random.choice(["Terrestre", "Aéreo"], size=n, p=[0.6, 0.4])  
distancia_km = np.random.normal(loc=300, scale=150, size=n).clip(50, 1200)
```

```
tiempo_entrega_hrs = np.where(
    tipo_transporte == "Terrestre",
    2 + 0.05 * distancia_km + np.random.normal(0, 1.5, n),
    1 + 0.02 * distancia_km + np.random.normal(0, 0.8, n)
)
```

```
costo_envio = np.where(
    tipo_transporte == "Terrestre",
    10 + 0.2 * distancia_km,
    50 + 0.8 * distancia_km
)
```

```
# Crear DataFrame
```

```
df_log = pd.DataFrame({
    "tipo_transporte": tipo_transporte,
    "distancia_km": distancia_km,
    "tiempo_entrega_hrs": tiempo_entrega_hrs,
    "costo_envio": costo_envio
})
```

```
# Introducir valores faltantes en distancia
```

```
df_log.loc[np.random.choice(n, 15), "distancia_km"] = np.nan
```

```
# Limpieza: Imputar con la mediana
```

```

print("=== VALORES FALTANTES ANTES ===")

print(df_log.isnull().sum())

mediana_dist = df_log["distancia_km"].median()

df_log["distancia_km"] = df_log["distancia_km"].fillna(mediana_dist)

print("\nValores faltantes DESPUÉS de imputar:", df_log.isnull().sum().sum())


# 2. Estadísticos de tiempo de entrega

media_tiempo = df_log["tiempo_entrega_hrs"].mean()

desv_tiempo = df_log["tiempo_entrega_hrs"].std()

print(f"\nMedia tiempo entrega: {media_tiempo:.2f} horas")

print(f"Desviación estándar: {desv_tiempo:.2f} horas\n")


# =====

# FASE 2: Análisis de Relaciones

# =====


corr_dist_tiempo, p_val = pearsonr(df_log["distancia_km"],
df_log["tiempo_entrega_hrs"])

print("=== CORRELACIÓN DISTANCIA - TIEMPO ===")

print(f"Coeficiente Pearson: {corr_dist_tiempo:.3f}")

print("¿Es fuerte? Sí, hay una correlación positiva considerable, pero depende del
transporte.\n")


# =====

```

```
# FASE 3: Comparativa de Modalidades
```

```
# =====
```

```
# 1. Promedios por transporte
```

```
resumen_transporte = df_log.groupby("tipo_transporte").agg(
```

```
    tiempo_promedio = ("tiempo_entrega_hrs", "mean"),
```

```
    costo_promedio = ("costo_envio", "mean")
```

```
)
```

```
print("=== RESUMEN POR TRANSPORTE ===")
```

```
print(resumen_transporte, "\n")
```

```
# 2. Prueba T de Student
```

```
tiempo_terrestre = df_log[df_log["tipo_transporte"] ==  
"Terrestre"]["tiempo_entrega_hrs"]
```

```
tiempo_aereo = df_log[df_log["tipo_transporte"] == "Aéreo"]["tiempo_entrega_hrs"]
```

```
t_stat, p_valor = ttest_ind(tiempo_terrestre, tiempo_aereo, equal_var=False)
```

```
print("=== PRUEBA DE HIPÓTESIS (T-TEST) ===")
```

```
print(f"Estadístico t: {t_stat:.2f}")
```

```
print(f"P-valor: {p_valor:.4f}")
```

```
print("Conclusión: Como  $p < 0.05$ , hay diferencia SIGNIFICATIVA en tiempos entre  
transporte.\n")
```

```
# =====
```

```
# FASE 4: Visualización
```

```
# =====
```

```
# 1. Gráfico de dispersión
```

```
plt.figure(figsize=(9,5))
```

```
sns.scatterplot(data=df_log, x="distancia_km", y="tiempo_entrega_hrs",
```

```
                hue="tipo_transporte", alpha=0.7, palette=["#2ecc71", "#3498db"])
```

```
plt.title("Distancia vs Tiempo de Entrega")
```

```
plt.xlabel("Distancia (km)")
```

```
plt.ylabel("Tiempo (horas)")
```

```
plt.savefig("scatter_logistica.png")
```

```
plt.show()
```

```
# 2. Boxplot tiempos
```

```
plt.figure(figsize=(7,5))
```

```
sns.boxplot(data=df_log, x="tipo_transporte", y="tiempo_entrega_hrs",  
            palette=["#2ecc71", "#3498db"])
```

```
plt.title("Comparación de Tiempos de Entrega")
```

```
plt.ylabel("Horas")
```

```
plt.savefig("boxplot_tiempos.png")
```

```
plt.show()
```

```
# =====
```

```
# RECOMENDACIÓN FINAL
```

```
# =====
```

```
print("=== RECOMENDACIÓN ===")
```

```
print("El transporte aéreo es mucho más rápido pero más costoso. Para mejorar la  
satisfacción")
```

```
print("del cliente, se recomienda migrar envíos prioritarios o de larga distancia al  
transporte aéreo,")
```

```
print("manteniendo terrestre para envíos cortos y de menor urgencia, equilibrando  
costo y velocidad.")
```

## Resultados:

```
103: sns.boxplot(horas)

PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS
PS C:\Users\guerr\OneDrive\Desktop\Practicas_Ciencia_Datos> python Practica_3.py

=== VALORES FALTANTES ANTES ===
tipo_transporte    0
distancia_km       15
tiempo_entrega_hrs  0
costo_envio         0
dtype: int64

Valores faltantes DESPUÉS de imputar: 0

Media tiempo entrega: 13.14 horas
Desviación estándar: 7.83 horas

=== CORRELACIÓN DISTANCIA - TIEMPO ===
Coeficiente Pearson: 0.729
¿Es fuerte? Sí, hay una correlación positiva considerable, pero depende del transporte.

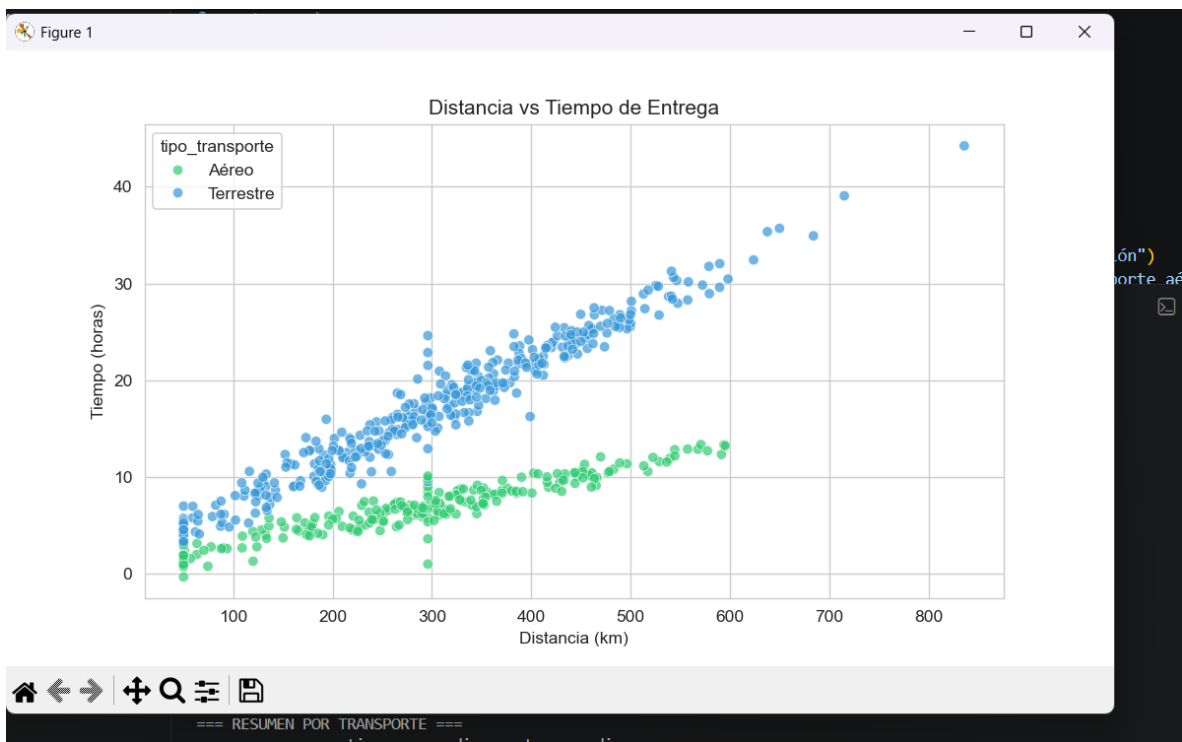
=== RESUMEN POR TRANSPORTE ===
                tiempo_promedio  costo_promedio
tipo_transporte
Aéreo                6.860464         283.113593
Terrestre            17.105524         70.408779

=== PRUEBA DE HIPÓTESIS (T-TEST) ===
Estadístico t: 23.94
P-valor: 0.0000
Conclusión: Como  $p < 0.05$ , hay diferencia SIGNIFICATIVA en tiempos entre transporte.

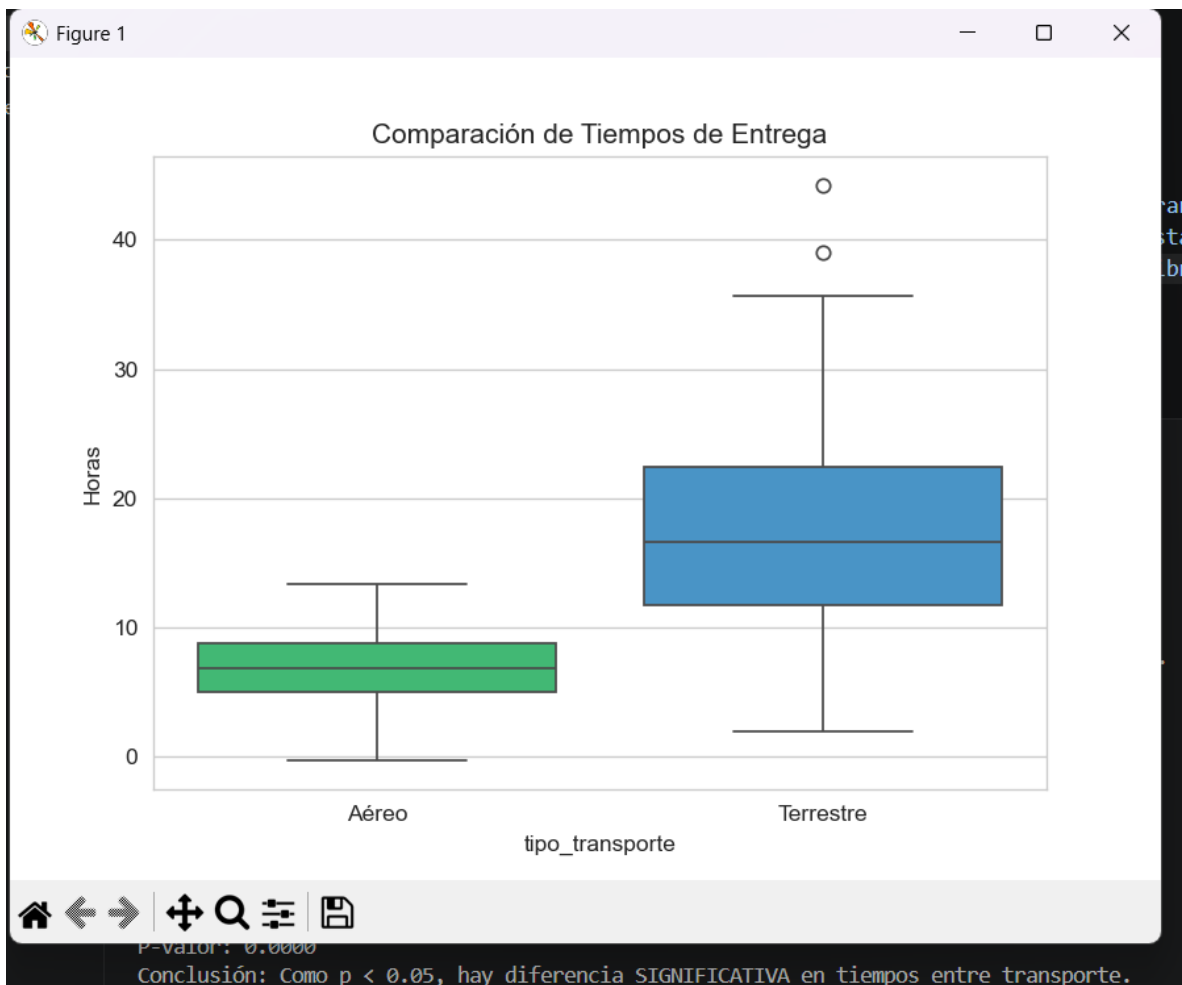
C:\Users\guerr\OneDrive\Desktop\Practicas_Ciencia_Datos\Practica_3.py:103: FutureWarning:
Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and set 'legend=False' for the same effect.

sns.boxplot(data=df_log, x="tipo_transporte", y="tiempo_entrega_hrs", palette=["#2ecc71", "#3498db"])

=== RECOMENDACIÓN ===
El transporte aéreo es mucho más rápido pero más costoso. Para mejorar la satisfacción
del cliente, se recomienda migrar envíos prioritarios o de larga distancia al transporte aéreo,
manteniendo terrestre para envíos cortos y de menor urgencia, equilibrando costo y velocidad.
PS C:\Users\guerr\OneDrive\Desktop\Practicas_Ciencia_Datos>
```







## Práctica 4: Reducción de Dimensionalidad en Monitoreo Industrial

### Código:

```
# =====  
  
# PRÁCTICA 4: REDUCCIÓN DE DIMENSIONALIDAD - PCA  
  
# =====  
  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
from sklearn.decomposition import PCA  
from sklearn.preprocessing import StandardScaler  
import seaborn as sns  
  
sns.set_style("whitegrid")  
np.random.seed(42) # Igual que en el código R original  
  
# =====  
  
# FASE 1: PREPARACIÓN DE DATOS  
  
# =====  
  
# 1. Generar dataset idéntico al ejemplo  
  
n = 200  
  
temp_nucleo = np.random.normal(100, 5, n)
```

```
temp_escape = temp_nucleo + np.random.normal(0, 1, n)
presion_interna = np.random.normal(50, 10, n)
vibracion_motor = (temp_nucleo * 0.5) + np.random.normal(0, 2, n)
flujo_gas = np.random.normal(20, 2, n)
oxigeno = np.random.normal(15, 1, n)
co2 = (temp_nucleo * 0.2) + np.random.normal(0, 0.5, n)
humedad = np.random.uniform(30, 40, n)
voltaje = np.random.normal(220, 2, n)
corriente = np.random.normal(15, 0.5, n)
ruido_db = (vibracion_motor * 1.2) + np.random.normal(0, 1, n)
eficiencia = 100 - (temp_nucleo * 0.1)
```

```
# Crear DataFrame
```

```
datos_sensores = pd.DataFrame({
    "temp_nucleo": temp_nucleo,
    "temp_escape": temp_escape,
    "presion_interna": presion_interna,
    "vibracion_motor": vibracion_motor,
    "flujo_gas": flujo_gas,
    "oxigeno": oxigeno,
    "co2": co2,
    "humedad": humedad,
    "voltaje": voltaje,
```

```
"corriente": corriente,  
"ruido_db": ruido_db,  
"eficiencia": eficiencia  
})
```

```
print("=== PRIMEROS 5 REGISTROS ===")  
print(datos_sensores.head(), "\n")
```

```
# 2. Correlación inicial
```

```
print("=== MATRIZ DE CORRELACIÓN ===")
```

```
matriz_corr = datos_sensores.corr().round(2)
```

```
print(matriz_corr)
```

```
print("\n☑ Se observa alta correlación entre: temp_nucleo - temp_escape,  
temp_nucleo - co2, vibracion - ruido\n")
```

```
# =====
```

```
# FASE 2: EJECUCIÓN DE PCA
```

```
# =====
```

```
# Estandarización (OBLIGATORIO, igual que scale. = TRUE en R)
```

```
escalador = StandardScaler()
```

```
datos_escalados = escalador.fit_transform(datos_sensores)
```

```
# Aplicar PCA
```

```

modelo_pca = PCA()

resultados_pca = modelo_pca.fit_transform(datos_escalados)

# =====

# FASE 3: SELECCIÓN DE COMPONENTES

# =====

# 1. Varianza explicada

varianza = modelo_pca.explained_variance_ratio_

varianza_acumulada = np.cumsum(varianza)

print("=== VARIANZA EXPLICADA ===")

for i, (v, va) in enumerate(zip(varianza, varianza_acumulada)):

    print(f'Componente {i+1}: {v:.3f} | Acumulada: {va:.3f}')

# 2. Scree Plot (Gráfico de sedimentación)

plt.figure(figsize=(8, 5))

plt.plot(range(1, len(varianza)+1), varianza, marker='o', linestyle='-',
color='darkblue')

plt.axhline(y=1/len(varianza), color='red', linestyle='--', label='Umbral Kaiser')

plt.title("Gráfico de Sedimentación (Scree Plot)")

plt.xlabel("Componentes Principales")

plt.ylabel("Proporción de Varianza")

plt.legend()

```

```
plt.savefig("scree_plot_sensores.png")
```

```
plt.show()
```

```
# 3. ¿Cuántos componentes para llegar al 85%?
```

```
num_componentes = np.argmax(varianza_acumulada >= 0.85) + 1
```

```
print(f"\n☑ Se necesitan {num_componentes} componentes para explicar el 85%  
de la varianza\n")
```

```
# 4. Cargas (Loadings) - Pesos en PC1 y PC2
```

```
cargas = pd.DataFrame(  
    modelo_pca.components_.T[:, :2],  
    columns=['PC1', 'PC2'],  
    index=datos_sensores.columns  
)  
.round(3)
```

```
print("=== CARGAS DE LOS COMPONENTES ===")
```

```
print(cargas)
```

```
print("\n💡 Interpretación:")
```

```
print("PC1 = ESTADO TÉRMICO (temperatura, vibración, CO2, ruido)")
```

```
print("PC2 = FACTORES INDEPENDIENTES (humedad, voltaje, corriente)\n")
```

```
# =====
```

```
# FASE 4: VISUALIZACIÓN (Biplot)
```

```
# =====
```

```

plt.figure(figsize=(10, 7))

# Puntos de observaciones
plt.scatter(resultados_pca[:, 0], resultados_pca[:, 1], alpha=0.6, color='gray')

# Vectores de variables
for variable in datos_sensores.columns:

    plt.arrow(0, 0, cargas.loc[variable, 'PC1'] * 4, cargas.loc[variable, 'PC2'] * 4,
              color='red', alpha=0.7, head_width=0.2)

    plt.text(cargas.loc[variable, 'PC1'] * 4.5, cargas.loc[variable, 'PC2'] * 4.5,
             variable, color='darkred', fontsize=9)

plt.title("Biplot: Sensores Industriales")
plt.xlabel(f"PC1 ({varianza[0]:.1%} varianza)")
plt.ylabel(f"PC2 ({varianza[1]:.1%} varianza)")
plt.axvline(x=0, color='black', linestyle='--', alpha=0.5)
plt.axhline(y=0, color='black', linestyle='--', alpha=0.5)
plt.savefig("biplot_sensores.png")
plt.show()

# =====

# CONCLUSIONES

# =====

print("=== CONCLUSIONES PARA GERENCIA DE TI ===")

```

```
print(f"1. Reducción: De 12 a {num_componentes} variables sin perder información crítica.")
```

```
print("2. Sensores redundantes: temp_escape, co2, ruido_db (se pueden calcular de otros).")
```

```
print("3. Impacto: Menor consumo de ancho de banda (~70% menos), procesamiento más rápido, sin saturación del PLC.")
```

## Resultados:

```
PS C:\Users\guerr\OneDrive\Desktop\Practicas_Ciencia_Datos> python Practica_4.py

=== PRIMEROS 5 REGISTROS ===
   temp_nucleo  temp_escape  presion_interna  vibracion_motor  flujo_gas  ...  humedad  voltaje  corriente  ruido_db  eficiencia
0    102.483571    102.841358     34.055723     52.755763    21.876568  ...    36.613706    222.985377    15.806139    64.362972    89.751643
1     99.308678     99.869463     44.006250     47.810009    18.967911  ...    31.851956    219.457753    15.657457    57.595249    90.069132
2    103.238443    104.321494     50.052437     53.358433    20.192242  ...    31.741093    219.957265    15.819982    63.975226    89.676156
3    107.615149    108.668951     50.469806     56.518850    19.075449  ...    30.983956    218.505577    15.371064    68.108175    89.238485
4     98.829233     97.451564     45.499345     50.241486    19.131008  ...    36.603027    215.151519    15.037717    60.810906    90.117077

[5 rows x 12 columns]

=== MATRIZ DE CORRELACIÓN ===
   temp_nucleo  temp_escape  presion_interna  vibracion_motor  flujo_gas  ...  humedad  voltaje  corriente  ruido_db  eficiencia
a
temp_nucleo      1.00      0.98      -0.13      0.77     -0.03  ...     -0.00     -0.03     -0.13     0.74     -1.00
0
temp_escape      0.98      1.00     -0.14      0.74     -0.06  ...     -0.03     -0.02     -0.10     0.71     -0.98
8
presion_interna  -0.13     -0.14      1.00     -0.03      0.03  ...     -0.06     -0.07     -0.04     -0.02      0.10
3
vibracion_motor   0.77      0.74     -0.03      1.00      0.09  ...      0.03      0.02     -0.12     0.97     -0.77
7
flujo_gas        -0.03     -0.06      0.03      0.09      1.00  ...      0.12      0.10      0.02      0.09      0.00
3
oxigeno          -0.09     -0.10     -0.06     -0.06      0.11  ...      0.04      0.09      0.04     -0.05      0.00
9
co2              0.88      0.86     -0.05      0.69     -0.03  ...     -0.05     -0.04     -0.10     0.68     -0.88
8
humedad          -0.00     -0.03     -0.06      0.03      0.12  ...      1.00      0.08     -0.13     0.03      0.00
0
voltaje          -0.03     -0.02     -0.07      0.02      0.10  ...      0.08      1.00      0.09      0.01      0.00
3
corriente        -0.13     -0.10     -0.04     -0.12      0.02  ...     -0.13      0.09      1.00     -0.13      0.10
3
ruido_db         0.74      0.71     -0.02      0.97      0.09  ...      0.03      0.01     -0.13      1.00     -0.77
4
eficiencia       -1.00     -0.98      0.13     -0.77      0.03  ...      0.00      0.03      0.13     -0.74      1.00
0

[12 rows x 12 columns]
```



```
PS C:\Users\guerr\OneDrive\Desktop\Practicas_Ciencia_Datos> python Practica_4.py
✅ Se observa alta correlación entre: temp_nucleo - temp_escape, temp_nucleo - co2, vibracion - ruido
```

=== VARIANZA EXPLICADA ===

|                |       |  |            |       |
|----------------|-------|--|------------|-------|
| Componente 1:  | 0.432 |  | Acumulada: | 0.432 |
| Componente 2:  | 0.109 |  | Acumulada: | 0.541 |
| Componente 3:  | 0.095 |  | Acumulada: | 0.637 |
| Componente 4:  | 0.087 |  | Acumulada: | 0.724 |
| Componente 5:  | 0.077 |  | Acumulada: | 0.800 |
| Componente 6:  | 0.071 |  | Acumulada: | 0.871 |
| Componente 7:  | 0.064 |  | Acumulada: | 0.935 |
| Componente 8:  | 0.047 |  | Acumulada: | 0.981 |
| Componente 9:  | 0.015 |  | Acumulada: | 0.996 |
| Componente 10: | 0.002 |  | Acumulada: | 0.998 |
| Componente 11: | 0.002 |  | Acumulada: | 1.000 |
| Componente 12: | 0.000 |  | Acumulada: | 1.000 |

✅ Se necesitan 6 componentes para explicar el 85% de la varianza

=== CARGAS DE LOS COMPONENTES ===

|                 | PC1    | PC2    |
|-----------------|--------|--------|
| temp_nucleo     | -0.427 | -0.026 |
| temp_escape     | -0.419 | -0.044 |
| presion_interna | 0.050  | -0.210 |
| vibracion_motor | -0.388 | 0.110  |
| flujo_gas       | 0.001  | 0.542  |
| oxigeno         | 0.047  | 0.437  |
| co2             | -0.395 | -0.063 |
| humedad         | 0.000  | 0.435  |
| voltaje         | 0.010  | 0.496  |
| corriente       | 0.070  | 0.060  |
| ruido_db        | -0.379 | 0.113  |
| eficiencia      | 0.427  | 0.026  |

```
PS C:\Users\guerr\OneDrive\Desktop\Practicas_Ciencia_Datos> python Practica_4.py
co2          -0.395 -0.063
humedad      0.000 0.435
voltaje      0.010 0.496
corriente    0.070 0.060
ruido_db     -0.379 0.113
eficiencia   0.427 0.026
```

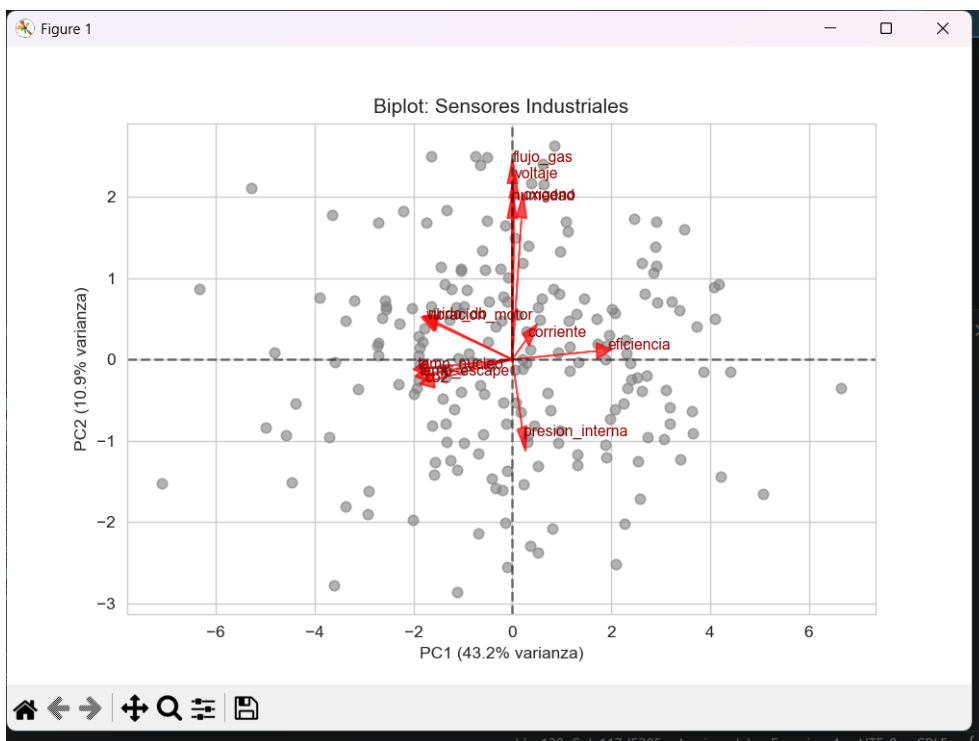
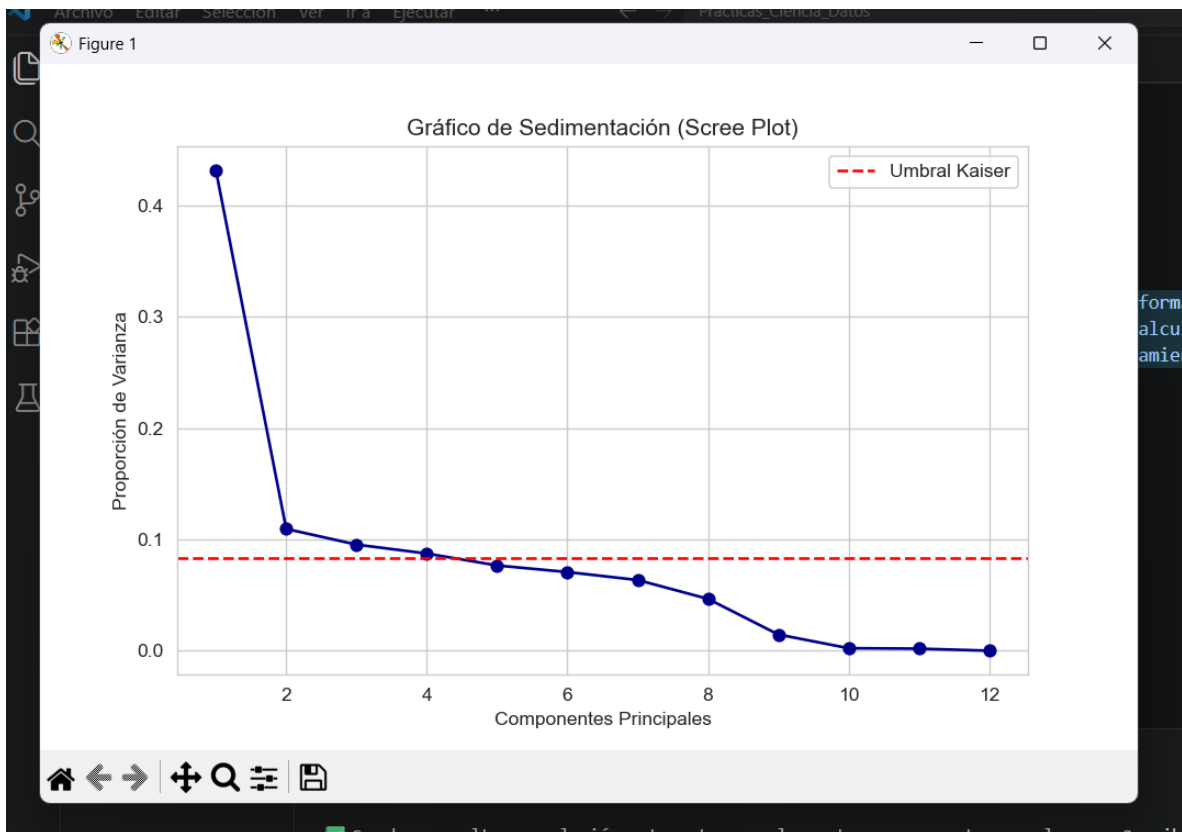
💡 Interpretación:

PC1 = ESTADO TÉRMICO (temperatura, vibración, CO2, ruido)  
PC2 = FACTORES INDEPENDIENTES (humedad, voltaje, corriente)

=== CONCLUSIONES PARA GERENCIA DE TI ===

1. Reducción: De 12 a 6 variables sin perder información crítica.
2. Sensores redundantes: temp\_escape, co2, ruido\_db (se pueden calcular de otros).
3. Impacto: Menor consumo de ancho de banda (~70% menos), procesamiento más rápido, sin saturación del PLC.

PS C:\Users\guerr\OneDrive\Desktop\Practicas\_Ciencia\_Datos> []



# Reporte Técnico: Reducción de Dimensionalidad en Monitoreo Industrial

## Código:

```
# Importar librerías necesarias

import numpy as np

import pandas as pd

from sklearn.decomposition import PCA

from sklearn.preprocessing import StandardScaler

import matplotlib.pyplot as plt

import os


# Crear carpeta para guardar todos los archivos (si no existe)

if not os.path.exists("Resultados_PCA"):

    os.makedirs("Resultados_PCA")


# -----

# 2. Metodología y Carga de Datos

# -----

np.random.seed(42)

n = 200


# Generar variables igual que en la práctica

temp1 = np.random.normal(100, 5, n)
```

```

temp2 = temp1 + np.random.normal(0, 1, n)

presion1 = np.random.normal(50, 10, n)

vibracion = (temp1 * 0.5) + np.random.normal(0, 2, n)


datos_sensores = pd.DataFrame({

    'temp_nucleo': temp1,

    'temp_escape': temp2,

    'presion_interna': presion1,

    'vibracion_motor': vibracion,

    'flujo_gas': np.random.normal(20, 2, n),

    'oxigeno': np.random.normal(15, 1, n),

    'co2': temp1 * 0.2 + np.random.normal(0, 0.5, n),

    'humedad': np.random.uniform(30, 40, n),

    'voltaje': np.random.normal(220, 2, n),

    'corriente': np.random.normal(15, 0.5, n),

    'ruido_db': vibracion * 1.2 + np.random.normal(0, 1, n),

    'eficiencia': 100 - (temp1 * 0.1)

})

```

```

# ☒ Guardar DATOS ORIGINALES

```

```

datos_sensores.to_csv("Resultados_PCA/1_datos_sensores_originales.csv",
index=False)

```

```

print("☒ Datos originales guardados")

```

```

# -----

# 3. Ejecución del Algoritmo PCA

# -----

escalador = StandardScaler()

datos_escalados = escalador.fit_transform(datos_sensores)


# ☒ Guardar DATOS ESCALADOS

pd.DataFrame(datos_escalados,
columns=datos_sensores.columns).to_csv("Resultados_PCA/2_datos_escalados.
csv", index=False)

print("☒ Datos escalados guardados")


pca_modelo = PCA()

componentes = pca_modelo.fit_transform(datos_escalados)


# ☒ Guardar VALORES DE LOS COMPONENTES PRINCIPALES (nuevas
variables)

pd.DataFrame(componentes, columns=[f'PC{i+1}' for i in
range(componentes.shape[1])]).to_csv("Resultados_PCA/3_componentes_princip
ales.csv", index=False)

print("☒ Componentes principales guardados")


# Resumen de varianza

varianza_explicada = pca_modelo.explained_variance_ratio_

varianza_acumulada = np.cumsum(varianza_explicada)

```

```
# ☒ Guardar RESUMEN DE VARIANZA
```

```
resumen_varianza = pd.DataFrame({  
    'Componente': [f'PC{i+1}' for i in range(len(varianza_explicada))],  
    'Varianza_Explicada': varianza_explicada.round(4),  
    'Varianza_Acumulada': varianza_acumulada.round(4)  
})  
  
resumen_varianza.to_csv("Resultados_PCA/4_resumen_varianza.csv",  
index=False)  
  
print("☒ Resumen de varianza guardado")
```

```
# -----
```

```
# 4.1 Selección de Componentes (Scree Plot)
```

```
# -----
```

```
plt.figure(figsize=(10, 6))  
  
plt.plot(range(1, len(varianza_explicada)+1), varianza_explicada, marker='o',  
linestyle='-', color='blue', label='Varianza individual')  
  
plt.plot(range(1, len(varianza_acumulada)+1), varianza_acumulada, marker='s',  
linestyle='--', color='green', label='Varianza acumulada')  
  
plt.axhline(y=0.85, color='red', linestyle=':', label='Umbral 85% varianza')  
  
plt.axhline(y=1/len(datos_sensores.columns), color='gray', linestyle='--',  
label='Criterio Kaiser')  
  
plt.title('Gráfico de Sedimentación (Scree Plot)')  
  
plt.xlabel('Número de Componentes Principales')  
  
plt.ylabel('Proporción de Varianza Explicada')  
  
plt.legend()
```

```
plt.grid(True)
```

```
# ☒ Guardar GRÁFICO DE SEDIMENTACIÓN
```

```
plt.savefig("Resultados_PCA/5_grafico_sedimentacion.png", dpi=300,  
bbox_inches='tight')
```

```
plt.close()
```

```
print("☒ Gráfico de sedimentación guardado")
```

```
# -----
```

```
# 4.2 Cargas de los Componentes (Loadings)
```

```
# -----
```

```
cargas = pd.DataFrame(  
    pca_modelo.components_.T,  
    index=datos_sensores.columns,  
    columns=[f'PC{i+1}' for i in range(pca_modelo.n_components_)])
```

```
# ☒ Guardar CARGAS (pesos de cada variable)
```

```
cargas.round(3).to_csv("Resultados_PCA/6_cargas_componentes.csv")
```

```
print("☒ Cargas de componentes guardadas")
```

```
# -----
```

```
# 4.3 Visualización del Biplot
```

```
# -----
```

```

plt.figure(figsize=(12, 8))

plt.scatter(componentes[:, 0], componentes[:, 1], alpha=0.6, color='lightblue')

for i, var in enumerate(datos_sensores.columns):

    plt.arrow(0, 0, cargas.iloc[i, 0]*5, cargas.iloc[i, 1]*5, color='red', alpha=0.7,
head_width=0.1)

    plt.text(cargas.iloc[i, 0]*5.2, cargas.iloc[i, 1]*5.2, var, color='darkred', fontsize=9)

plt.title('Biplot: Componentes Principales vs Variables Originales')

plt.xlabel(f'PC1 ({varianza_explicada[0]:.1%} varianza)')

plt.ylabel(f'PC2 ({varianza_explicada[1]:.1%} varianza)')

plt.grid(True)

# ☒ Guardar BILOT

plt.savefig("Resultados_PCA/7_biplot.png", dpi=300, bbox_inches='tight')

plt.close()

print("☒ Biplot guardado")

print("\n 🎉 ¡Todo ha sido guardado correctamente en la carpeta
'Resultados_PCA!'")

```



## Resultados:

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS

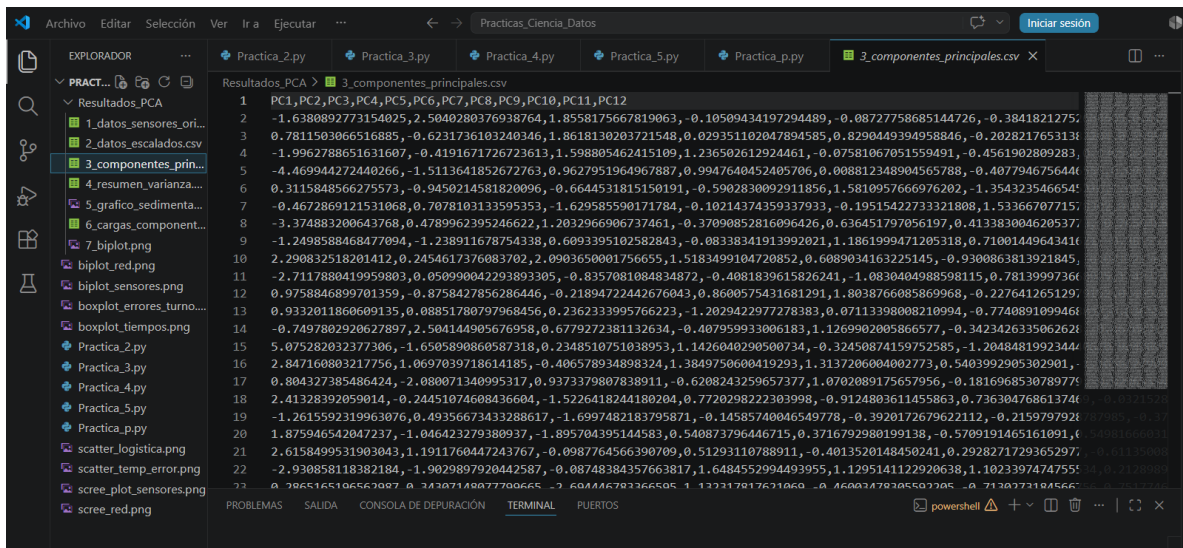
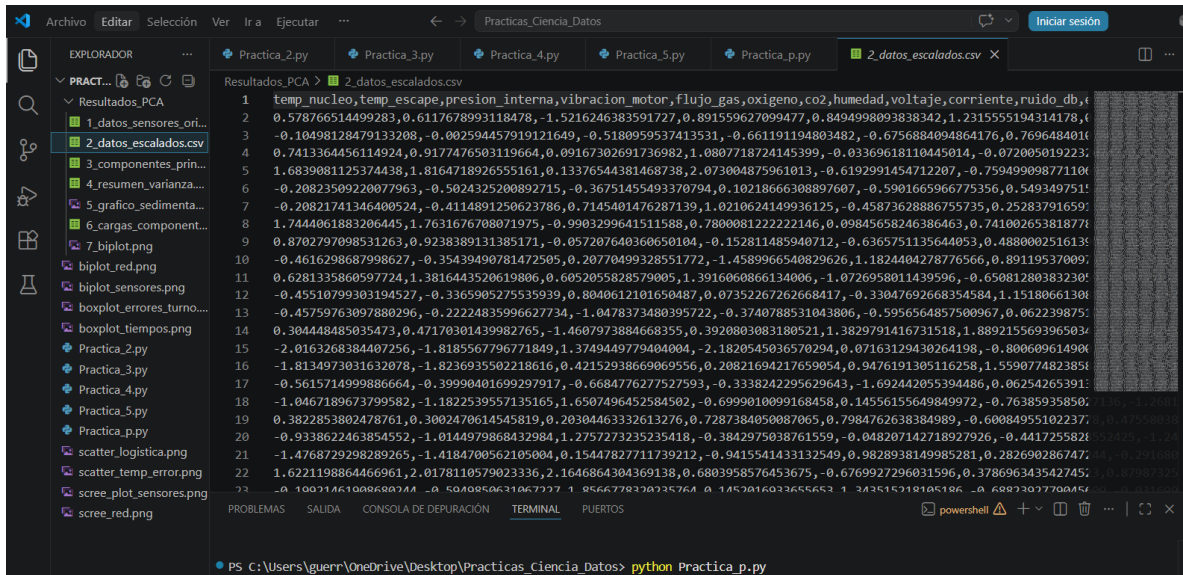
• PS C:\Users\guerr\OneDrive\Desktop\Practicas_Ciencia_Datos> python Practica_p.py
• PS C:\Users\guerr\OneDrive\Desktop\Practicas_Ciencia_Datos> python Practica_p.py
  ✓ Datos originales guardados
  ✓ Datos escalados guardados
  ✓ Componentes principales guardados
  ✓ Resumen de varianza guardado
  ✓ Gráfico de sedimentación guardado
  ✓ Cargas de componentes guardadas
  ✓ Biplot guardado

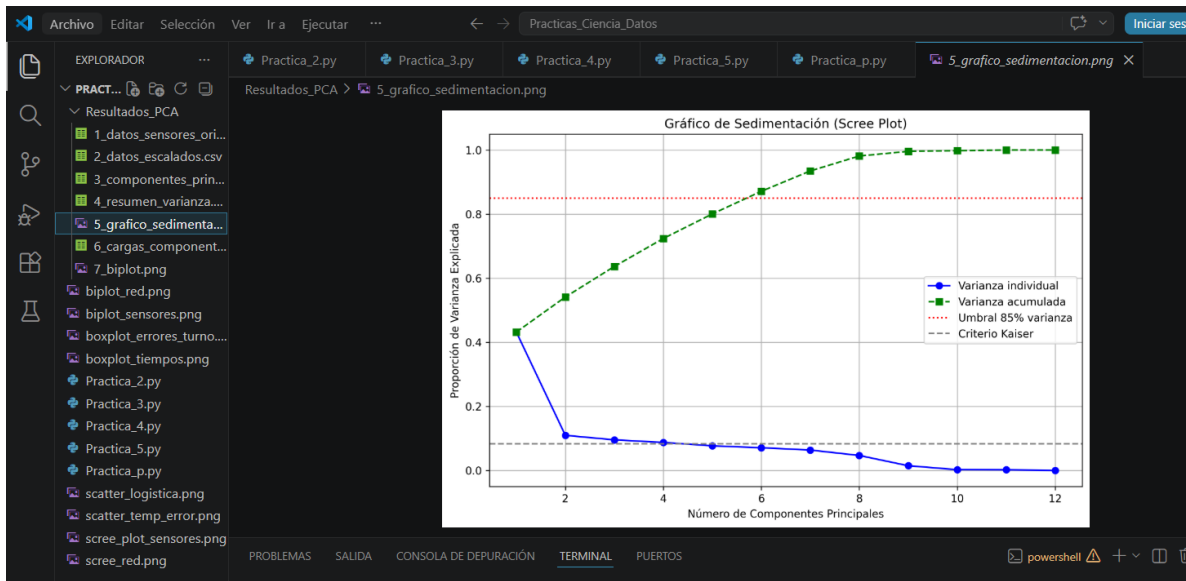
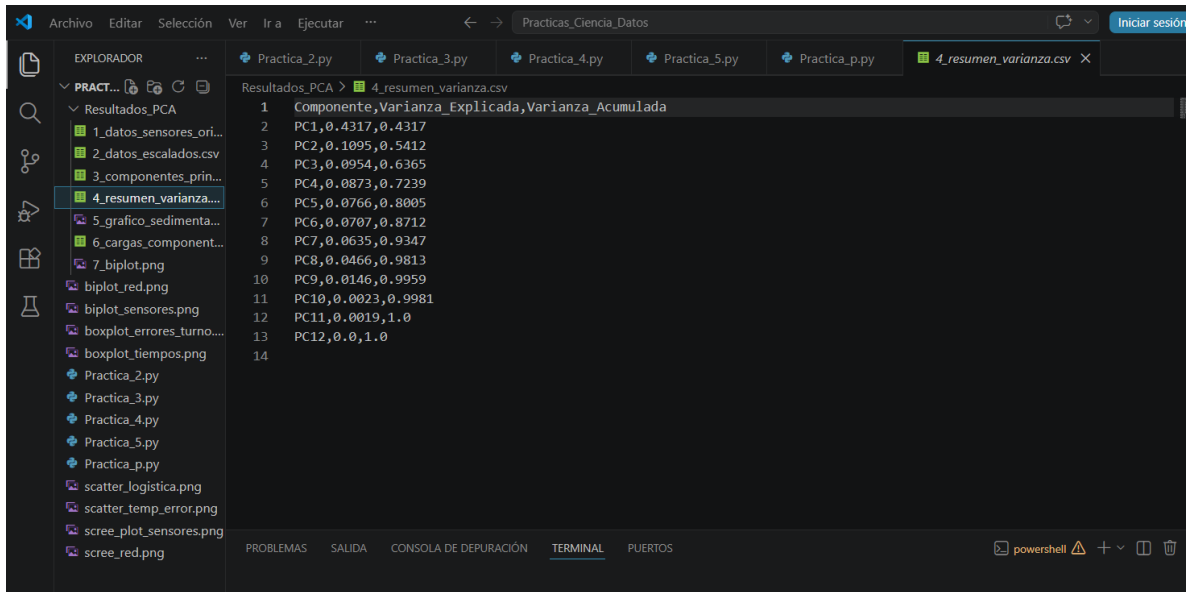
🎉 ¡Todo ha sido guardado correctamente en la carpeta 'Resultados_PCA'!
○ PS C:\Users\guerr\OneDrive\Desktop\Practicas_Ciencia_Datos> []
```

```
Ver  Ir a  Ejecutar  ...  <  >  Practicas_Ciencia_Datos  Iniciar sesión

Practica_2.py  Practica_3.py  Practica_4.py  Practica_5.py  Practica_p.py  1_datos_sensores_originales.csv  ...

Resultados_PCA > 1_datos_sensores_originales.csv
1  temp_nucleo,temp_escape,presion_interna,vibracion_motor,flujo_gas,oxigeno,co2,humedad,voltaje,corriente,ruido_db,t
2  102.48357076505616,102.84135812540444,34.05572341205633,52.75576261581878,21.876567611951995,16.399355436586003,20
3  99.30867849414408,99.86946302051231,44.00624977046227,47.810008598716784,18.967910543565253,15.924633682912768,19
4  103.23844269050346,104.32149393367874,50.05243699718183,53.35843318546305,20.192241553881967,15.059630369920175,20
5  107.61514928204012,108.66895133407502,50.46980593764742,56.51885035862996,19.075449422589916,14.353063222294427,20
6  98.82923312638331,97.45156375842622,45.49934528520756,50.24148636963906,19.13100754513537,15.69822331361359,19.790
7  98.8293152152541,97.89149017533897,56.228499323474985,53.16824923274318,19.38165575306272,15.39348538542175,19.788
8  107.89606407753696,108.41109934474562,39.32379570617405,52.40045364056134,20.444267543267426,15.895193220027732,20
9  103.83717364576455,104.35095959667676,48.576205149787064,49.42927741625944,19.042502756673045,15.63517180168197,20
10 97.65262807032524,98.16767575663128,51.20295631711899,45.26887353735406,22.511512251147042,16.049552715319336,18.8
11 102.71280021792982,106.56553170858454,55.144388340588749,54.34848873194328,18.210785395560993,14.464764788439432,20
12 97.68291153593769,98.25380204663085,57.1161487808889,50.150187080676965,19.626256711677282,16.317394065634325,18.9
13 97.67135123214871,98.80691687232931,38.75357908162131,48.72450627425344,19.12053788345165,15.1975996046924,19.187
14 101.20981135783018,102.16381312132337,34.65885829264378,51.16484293155473,22.893955768707464,17.075260872625265,19
15 90.43359877671101,91.0849900280168,62.77676821898509,42.96582129375875,20.393109553023148,14.310812181910432,18.5
16 91.37541083743484,91.0601415927945,53.323140119795916,50.579209377951074,22.06368907893727,16.735963803165248,18.1
17 97.18856235379513,97.9475315742884,42.51513463443446,48.852723540848025,17.028879253926057,15.197910783462648,20.0
18 94.93584439832789,94.16301918379031,65.51151975522524,47.68671178837373,20.534100531738517,14.348581996385551,18.6
19 101.57123666297637,101.33441805623636,51.156746342928585,52.23715157928557,21.779261591246875,14.516114165945678,20.45378444137
20 95.45987962239394,94.97451607456483,61.792971840638266,48.6919582746704,20.16456797855085,14.67965269180568,18.67430239784891,19
21 92.93848149332354,93.02035563270987,50.67518481410109,46.91700879522003,22.130960750130704,15.424165946401917,19.602708029514
22 107.32824384460777,109.64290241128128,70.60747924881987,52.08317301141326,18.965423099799256,15.522835488035499,20.87184955940
23 98.87111840756733,97.00385330407558,67.55340847443304,50.379405063055656,22.81860488037116,14.476790090606142,19.07113405403609
```





Archivo Editar Selección Ver Ir a Ejecutar ... Prácticas\_Ciencia\_Datos Iniciar sesión

EXPLORADOR ...

- PRACT...
- Resultados\_PCA
  - 1\_datos\_sensores\_ori...
  - 2\_datos\_escalados.csv
  - 3\_componentes\_prin...
  - 4\_resumen\_varianza...
  - 5\_grafico\_sedimenta...
  - 6\_cargas\_component...
  - 7\_biplot.png
  - biplot\_red.png
  - biplot\_sensores.png
  - boxplotErrores\_turno...
  - boxplot\_tiempos.png
- Practica\_2.py
- Practica\_3.py
- Practica\_4.py
- Practica\_5.py
- Practica\_p.py
- scatter\_logistica.png
- scatter\_temp\_error.png
- screeplot\_sensores.png
- screep\_red.png

Resultados\_PCA > 6\_cargas\_componentes.csv

```
1 ,PC1,PC2,PC3,PC4,PC5,PC6,PC7,PC8,PC9,PC10,PC11,PC12
2 temp_nucleo,-0.427,-0.026,0.049,-0.061,0.021,-0.005,0.028,-0.24,-0.256,-0.275,-0.334,0.707
3 temp_escape,-0.419,-0.044,0.091,-0.062,0.009,0.008,0.04,-0.262,-0.338,0.579,0.539,0.0
4 presion_interna,0.05,-0.21,-0.393,0.709,-0.055,0.37,0.33,-0.204,-0.078,-0.001,-0.008,-0.0
5 vibracion_motor,-0.388,0.11,-0.079,0.161,-0.029,0.002,0.01,0.526,-0.039,-0.508,0.517,0.0
6 flujo_gas,0.001,0.542,-0.199,0.426,-0.006,-0.515,-0.393,-0.256,-0.021,0.018,0.008,0.0
7 oxigeno,0.047,0.437,0.176,0.045,0.811,0.281,0.192,-0.015,-0.008,-0.003,0.007,0.0
8 co2,-0.395,-0.063,0.056,0.025,0.031,0.022,0.032,-0.299,0.859,-0.008,0.08,0.0
9 humedad,0.0,0.435,-0.469,-0.386,-0.217,-0.116,0.613,-0.061,0.04,0.013,0.011,0.0
10 voltaje,0.01,0.496,0.285,0.013,-0.499,0.615,-0.195,-0.078,0.006,-0.007,-0.012,0.0
11 corriente,0.07,0.06,0.666,0.313,-0.2,-0.355,0.531,0.031,-0.006,-0.015,-0.022,-0.0
12 ruido_db,-0.379,0.113,-0.087,0.173,-0.017,0.01,0.007,0.576,0.083,0.504,-0.461,-0.0
13 eficiencia,0.427,0.026,-0.049,0.061,-0.021,0.005,-0.028,0.24,0.256,0.275,0.334,0.707
14
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS powershell

Archivo Editar Selección Ver Ir a Ejecutar ... Prácticas\_Ciencia\_Datos Iniciar sesión

EXPLORADOR ...

- PRACT...
- Resultados\_PCA
  - 1\_datos\_sensores\_ori...
  - 2\_datos\_escalados.csv
  - 3\_componentes\_prin...
  - 4\_resumen\_varianza...
  - 5\_grafico\_sedimenta...
  - 6\_cargas\_component...
  - 7\_biplot.png
  - biplot\_red.png
  - biplot\_sensores.png
  - boxplotErrores\_turno...
  - boxplot\_tiempos.png
- Practica\_2.py
- Practica\_3.py
- Practica\_4.py
- Practica\_5.py
- Practica\_p.py
- scatter\_logistica.png
- scatter\_temp\_error.png
- screeplot\_sensores.png
- screep\_red.png

Resultados\_PCA > 7\_biplot.png

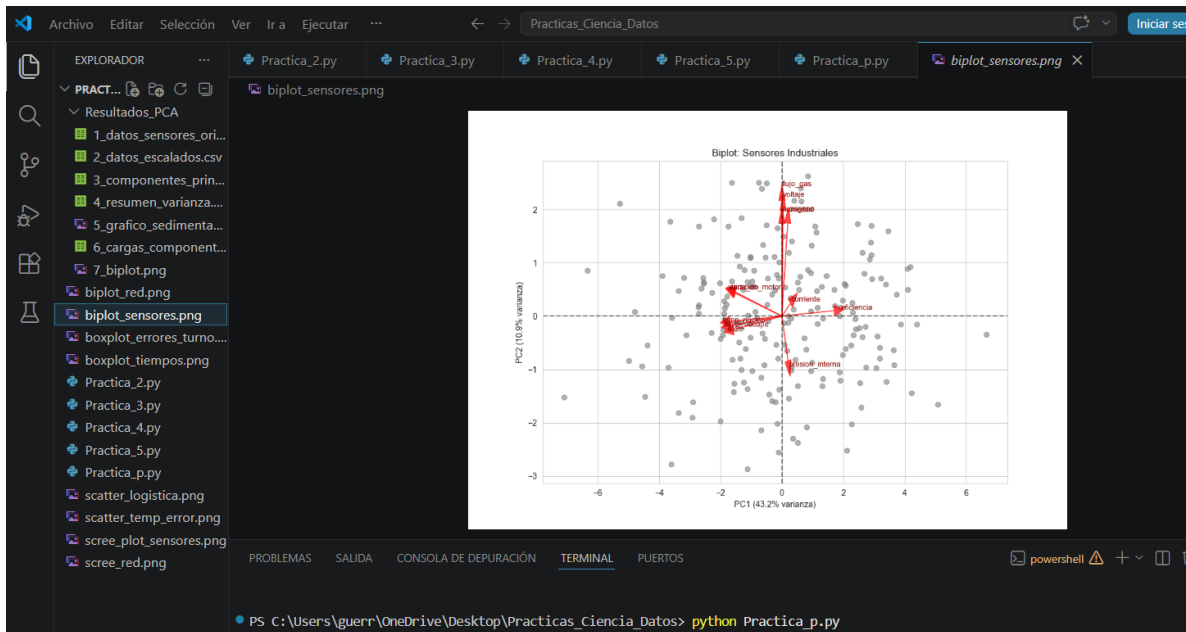
PC2 (15.9% varianza)

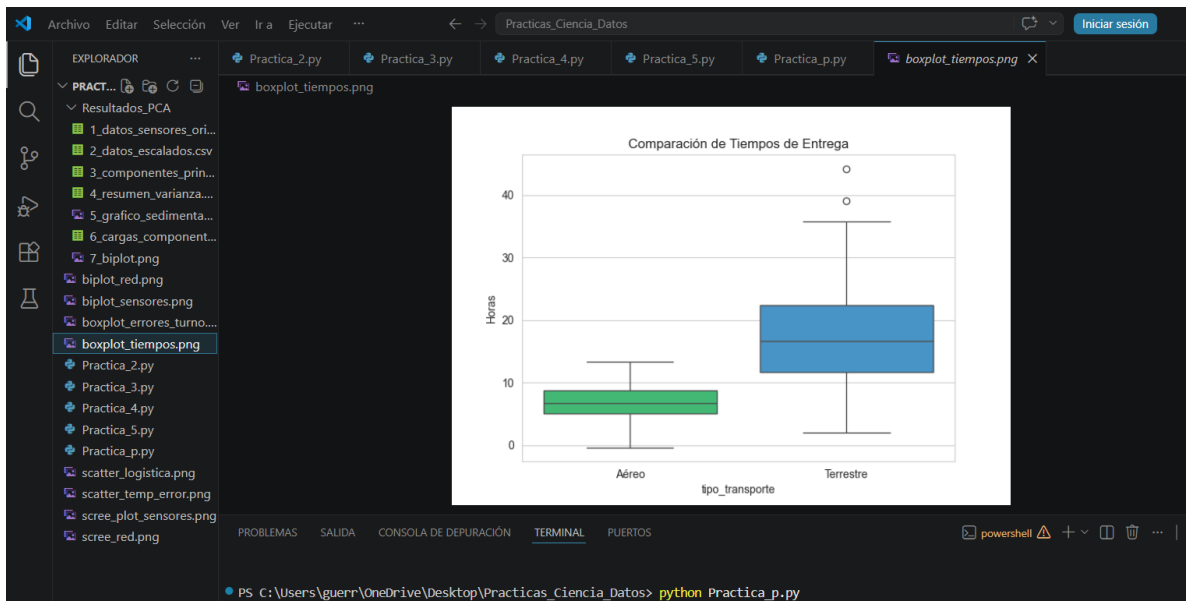
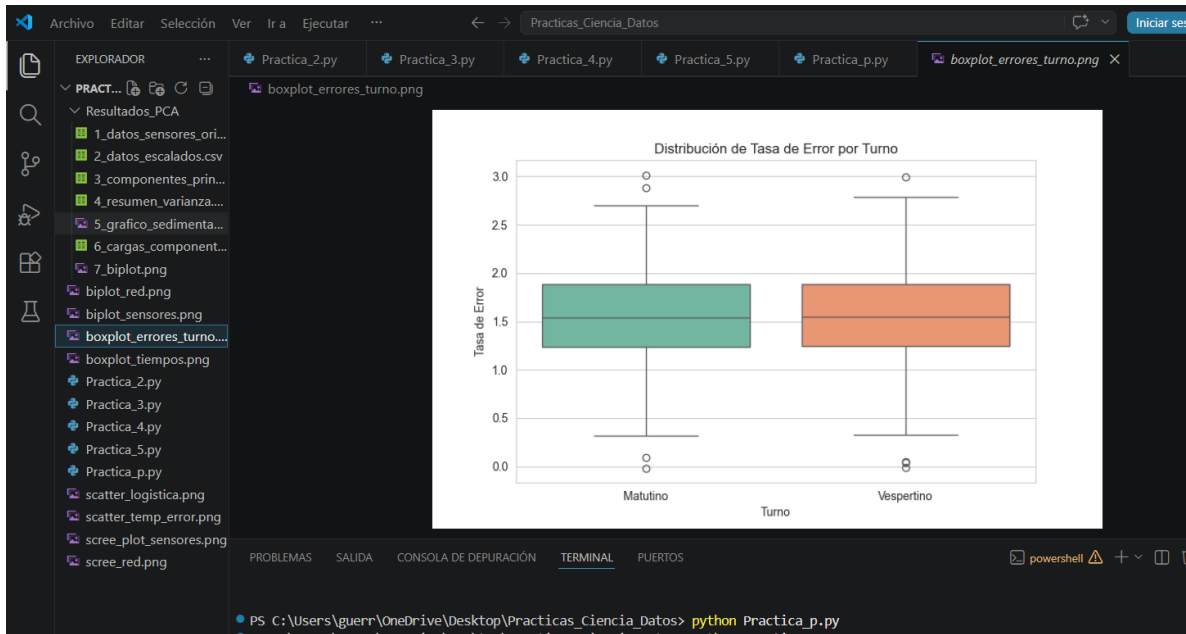
PC1 (43.2% varianza)

flujo\_gas  
voltaje  
eficiencia  
corriente  
presion\_interna  
vibracion\_motor  
co2  
oxigeno  
humedad  
temp\_nucleo  
temp\_escape  
ruido\_db

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS powershell

```
PS C:\Users\guerr\OneDrive\Desktop\Practicas_Ciencia_Datos> python Practica_p.py
PS C:\Users\guerr\OneDrive\Desktop\Practicas_Ciencia_Datos> python Practica_p.py
```





## Preguntas:

- **datos\_sensores\_originales.csv** → Los datos tal como los generamos, con las 12 variables.
- **datos\_escalados.csv** → Los datos después de estandarizarlos (media 0, desviación 1), que es lo que usa el algoritmo.
- **componentes\_principales.csv** → Los nuevos valores reducidos (las nuevas variables PC1, PC2, ..., PC12). Aquí es donde se ve la reducción.
- **resumen\_varianza.csv** → Cuánta información explica cada componente y la acumulada (para ver el 85%).
- **grafico\_sedimentacion.png** → La gráfica para identificar el "codo".
- **cargas\_componentes.csv** → Qué tanto aporta cada sensor a cada componente (para saber cuáles son redundantes).
- **biplot.png** → La gráfica con las variables y observaciones.

## Unidad 2: Tratamiento de Datos - Ciencia de Datos"

### Código:

```
# =====  
  
# PRÁCTICA: REDUCCIÓN DE DIMENSIONALIDAD - METAL-TECH  
  
# FECHA: 2026-06-04 20:20:21  
  
# =====  
  
# -----  
  
# 1. IMPORTAR LIBRERÍAS  
  
# -----  
  
import numpy as np  
  
import pandas as pd  
  
import matplotlib.pyplot as plt  
  
from sklearn.decomposition import PCA  
  
from sklearn.preprocessing import StandardScaler  
  
import os # Esta librería sirve para crear la carpeta donde guardaremos todo  
  
# -----  
  
# CREAR CARPETA DESTINO  
  
# -----  
  
# Creamos una carpeta llamada "Resultados_PCA" si no existe aún  
  
if not os.path.exists("Resultados_PCA"):  
    os.makedirs("Resultados_PCA")
```



```
print("☑ Carpeta 'Resultados_PCA' creada o ya existente.")
```

```
# =====
```

```
# 2. METODOLOGÍA Y CARGA DE DATOS
```

```
# =====
```

```
print("\n ♦ PASO 2: Generando datos de sensores...")
```

```
np.random.seed(42) # Equivalente a set.seed(42) en R
```

```
n = 200
```

```
# Generación de variables correlacionadas igual que en tu código R
```

```
temp1 = np.random.normal(loc=100, scale=5, size=n)
```

```
temp2 = temp1 + np.random.normal(loc=0, scale=1, size=n)
```

```
presion1 = np.random.normal(loc=50, scale=10, size=n)
```

```
vibracion = (temp1 * 0.5) + np.random.normal(loc=0, scale=2, size=n)
```

```
# Crear DataFrame con los 12 sensores
```

```
datos_sensores = pd.DataFrame({
```

```
    'temp_nucleo': temp1,
```

```
    'temp_escape': temp2,
```

```
    'presion_interna': presion1,
```

```
    'vibracion_motor': vibracion,
```

```
    'flujo_gas': np.random.normal(loc=20, scale=2, size=n),
```

```

'oxigeno': np.random.normal(loc=15, scale=1, size=n),
'co2': temp1 * 0.2 + np.random.normal(loc=0, scale=0.5, size=n),
'humedad': np.random.uniform(low=30, high=40, size=n),
'voltaje': np.random.normal(loc=220, scale=2, size=n),
'corriente': np.random.normal(loc=15, scale=0.5, size=n),
'ruido_db': vibracion * 1.2 + np.random.normal(loc=0, scale=1, size=n),
'eficiencia': 100 - (temp1 * 0.1)
})

# Mostrar en pantalla (equivalente a head())
print(datos_sensores.head())

# 📁 GUARDADO 1: Datos originales
datos_sensores.to_csv("Resultados_PCA/01_datos_originales.csv", index=False)
print(" ✅ Guardado: 01_datos_originales.csv")

# =====

# 3. EJECUCIÓN DEL ALGORITMO PCA

# =====

print("\n 💡 PASO 3: Ejecutando PCA...")

# Estandarizar datos (center=TRUE, scale.=TRUE)
escalador = StandardScaler()

```

```
datos_escalados = escalador.fit_transform(datos_sensores)
```

```
#  GUARDADO 2: Datos escalados/estandarizados
```

```
pd.DataFrame(datos_escalados,  
columns=datos_sensores.columns).to_csv("Resultados_PCA/02_datos_escalados  
.csv", index=False)
```

```
print("  Guardado: 02_datos_escalados.csv")
```

```
# Aplicar modelo PCA
```

```
pca_modelo = PCA()
```

```
pca_modelo.fit(datos_escalados)
```

```
# Resumen de varianza (equivalente a summary(pca_modelo))
```

```
varianza_explicada = pca_modelo.explained_variance_ratio_
```

```
varianza_acumulada = np.cumsum(varianza_explicada)
```

```
resumen_varianza = pd.DataFrame({
```

```
    'Varianza_Explicada_Individual': varianza_explicada.round(4),
```

```
    'Varianza_Explicada_Acumulada': varianza_acumulada.round(4)
```

```
}, index=[f'PC{i+1}' for i in range(len(varianza_explicada))])
```

```
print(resumen_varianza)
```

```
#  GUARDADO 3: Resumen de varianza
```

```
resumen_varianza.to_csv("Resultados_PCA/03_resumen_varianza.csv")
```

```
print(" ☒ Guardado: 03_resumen_varianza.csv")
```

```
# Guardamos también los valores calculados de cada componente para usarlos después
```

```
componentes = pca_modelo.transform(datos_escalados)
```

```
pd.DataFrame(componentes, columns=[f'PC{i+1}' for i in range(componentes.shape[1])]).to_csv("Resultados_PCA/04_valores_componentes.csv", index=False)
```

```
print(" ☒ Guardado: 04_valores_componentes.csv")
```

```
# =====
```

```
# 4. ANÁLISIS DE RESULTADOS
```

```
# =====
```

```
# -----
```

```
# 4.1 Gráfico de Sedimentación
```

```
# -----
```

```
print("\n ♦ PASO 4.1: Generando Gráfico de Sedimentación...")
```

```
plt.figure(figsize=(10, 6))
```

```
plt.plot(range(1, len(varianza_explicada)+1), varianza_explicada, marker='o', linestyle='-', color='blue', label='Varianza Individual')
```

```
plt.plot(range(1, len(varianza_acumulada)+1), varianza_acumulada, marker='s', linestyle='--', color='green', label='Varianza Acumulada')
```

```

# Líneas de referencia solicitadas

plt.axhline(y=1/len(datos_sensores.columns), color='red', linestyle='--',
label='Criterio Kaiser (valor > 1)')

plt.axhline(y=0.85, color='orange', linestyle=':', label='Objetivo 85% Varianza')


plt.title('Gráfico de Sedimentación (Scree Plot)')

plt.xlabel('Número de Componentes Principales')

plt.ylabel('Proporción de Varianza')

plt.legend()

plt.grid(True)


# 📁 GUARDADO 4: Gráfico de sedimentación

plt.savefig("Resultados_PCA/05_grafico_sedimentacion.png", dpi=300,
bbox_inches='tight')

plt.close() # Cerramos la figura para no mezclar con la siguiente

print(" ☒ Guardado: 05_grafico_sedimentacion.png")


# -----

# 4.2 Cargas de los Componentes (Loadings)

# -----

print("\n ◆ PASO 4.2: Analizando Cargas...")


# Equivalente a pca_modelo$rotation

cargas = pd.DataFrame(

```

```
pca_modelo.components_.T,  
index=datos_sensores.columns,  
columns=[f'PC{i+1}' for i in range(pca_modelo.n_components_)])
```

```
# Mostrar solo los primeros 2 redondeados a 3 decimales
```

```
print(cargas[['PC1', 'PC2']].round(3))
```

```
#  GUARDADO 5: Cargas completas
```

```
cargas.round(3).to_csv("Resultados_PCA/06_cargas_componentes.csv")
```

```
print("  Guardado: 06_cargas_componentes.csv")
```

```
# -----
```

```
# 4.3 Visualización del Biplot
```

```
# -----
```

```
print("\n  PASO 4.3: Generando Biplot...")
```

```
plt.figure(figsize=(12, 8))
```

```
# Graficar puntos de datos
```

```
plt.scatter(componentes[:, 0], componentes[:, 1], alpha=0.6, color='lightblue')
```

```
# Graficar vectores de las variables originales
```

```
escala = 5 # Factor para que se vean bien las flechas
```

```
for i, var in enumerate(datos_sensores.columns):
```

```
    plt.arrow(0, 0, cargas.iloc[i, 0]*escala, cargas.iloc[i, 1]*escala, color='red',  
alpha=0.7, head_width=0.1)
```

```
    plt.text(cargas.iloc[i, 0]*escala*1.1, cargas.iloc[i, 1]*escala*1.1, var,  
color='darkred', fontsize=9)
```

```
plt.title('Biplot de Sensores Industriales')
```

```
plt.xlabel(f'PC1 ({varianza_explicada[0]:.1%} varianza)')
```

```
plt.ylabel(f'PC2 ({varianza_explicada[1]:.1%} varianza)')
```

```
plt.grid(True)
```

```
# 📁 GUARDADO 6: Biplot
```

```
plt.savefig("Resultados_PCA/07_biplot.png", dpi=300, bbox_inches='tight')
```

```
plt.close()
```

```
print(" ☒ Guardado: 07_biplot.png")
```

```
# =====
```

```
# FINALIZACIÓN
```

```
# =====
```

```
print("\n 🎉 ¡PROCESO FINALIZADO!")
```

```
print("Todos los archivos están guardados en la carpeta 'Resultados_PCA'")
```

## Resultados:

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS
PS C:\Users\guerr\OneDrive\Desktop\Practicas_Ciencia_Datos> python Practica_PCA.PY
✓ Carpeta 'Resultados_PCA' creada o ya existente.

♦ PASO 2: Generando datos de sensores...
temp_nucleo temp_escape presion_interna vibracion_motor flujo_gas ... humedad voltaje corriente ruido_db eficiencia
0 102.483571 102.841358 34.055723 52.755763 21.876568 ... 36.613706 222.985377 15.806139 64.362972 89.751643
1 99.308678 99.869463 44.006250 47.810009 18.967911 ... 31.851956 219.457753 15.657457 57.595249 90.069132
2 103.238443 104.321494 50.052437 53.358433 20.192242 ... 31.741093 219.957265 15.819982 63.975226 89.676156
3 107.615149 108.668951 50.469806 56.518850 19.075449 ... 30.983956 218.505577 15.371064 68.108175 89.238485
4 98.829233 97.451564 45.499345 50.241486 19.131008 ... 36.603027 215.151519 15.037717 60.810906 90.117077

[5 rows x 12 columns]
✓ Guardado: 01_datos_originales.csv

♦ PASO 3: Ejecutando PCA...
✓ Guardado: 02_datos_escalados.csv
Varianza_Explicada_Individual Varianza_Explicada_Acumulada
PC1 0.4317 0.4317
PC2 0.1095 0.5412
PC3 0.0954 0.6365
PC4 0.0873 0.7239
PC5 0.0766 0.8005
PC6 0.0707 0.8712
PC7 0.0635 0.9347
PC8 0.0466 0.9813
PC9 0.0146 0.9959
PC10 0.0023 0.9981
PC11 0.0019 1.0000
PC12 0.0000 1.0000
✓ Guardado: 03_resumen_varianza.csv
✓ Guardado: 04_valores_componentes.csv
```



♦ PASO 4.1: Generando Gráfico de Sedimentación...

✓ Guardado: 05\_grafico\_sedimentacion.png

♦ PASO 4.2: Analizando Cargas...

|                 | PC1    | PC2    |
|-----------------|--------|--------|
| temp_nucleo     | -0.427 | -0.026 |
| temp_escape     | -0.419 | -0.044 |
| presion_interna | 0.050  | -0.210 |
| vibracion_motor | -0.388 | 0.110  |
| flujo_gas       | 0.001  | 0.542  |
| oxigeno         | 0.047  | 0.437  |
| co2             | -0.395 | -0.063 |
| humedad         | 0.000  | 0.435  |
| voltaje         | 0.010  | 0.496  |
| corriente       | 0.070  | 0.060  |
| ruido_db        | -0.379 | 0.113  |
| eficiencia      | 0.427  | 0.026  |

✓ Guardado: 06\_cargas\_componentes.csv

♦ PASO 4.3: Generando Biplot...

✓ Guardado: 07\_biplot.png

🎉 ¡PROCESO FINALIZADO!

Todos los archivos están guardados en la carpeta 'Resultados PCA'

PS C:\Users\guerr\OneDrive\Desktop\Practicas\_Ciencia\_Datos> █

Archivo Editar Selección Ver Ir a Ejecutar ... Practicas\_Ciencia\_Datos Iniciar sesión

EXPLORADOR ... Practica\_2.py Practica\_3.py Practica\_4.py Practica\_5.py Practica\_p.py Practica\_PCA.py 2\_datos\_escalados.csv

Resultados\_PCA > 2\_datos\_escalados.csv

```
1 temp_nucleo,temp_escape,presion_interna,vibracion_motor,flujo_gas,oxigeno,co2,humedad,voltaje,corriente,ruido_db,
2 0.578766514499283,0.6117678993118478,-1.5216246383591727,0.891559627099477,0.849498093838342,1.2315555194314178,
3 -0.10498128479133208,-0.002594457919121649,-0.518095937413531,-0.661191194803482,-0.6756884094864176,0.7696484016
4 0.7413364456114924,0.9177476503119664,0.09167302691736982,1.0807718724145399,-0.03369618110445014,-0.072005019223
5 1.6839081125374438,1.8164718926555161,0.13376544381468738,2.073004875961013,-0.6102991454712207,0.75949908771108
6 -0.20823509220077963,-0.5024325200892715,-0.36751455493370794,0.10218666308897607,-0.5901665966775356,0.549349751
7 -0.20821741346400524,-0.4114891250623786,0.7145401476287139,1.0210624149936125,-0.45873628886755735,0.25283791659
8 1.7444061883206445,1.7631676708071975,-0.9903299641511588,0.7800081222222146,0.09845658246386463,0.74100265381877
9 0.8702797098531263,0.9238389131385171,-0.057207640360650104,-0.152811485940712,-0.6365751135644053,0.48800251613
10 -0.4616298687998627,-0.35439490781472505,0.20770499328551772,-1.4589966540829626,1.1824404278776566,0.89119537009
11 0.6781335860597724,1.3816443520619806,0.6052055828579005,1.3916060866134006,-1.0726958011439596,-0.65081280383230
12 -0.45510799303194527,-0.3365985275535939,0.8040612101650487,0.07352267262668417,-0.33047692668354584,1.1518066130
13 -0.45759763097880296,-0.22224835996627734,-1.0478373480395722,-0.3740788531043806,-0.5956564857500967,0.062239875
14 0.304448485035473,0.47170301439982765,-1.4607973884668355,0.3020803083180521,1.3829791416731518,1.88921556930650
15 -2.0163268384407256,-1.8185567796771849,1.374844977940084,-2.1820545036570204,0.07163129430264198,-0.80060961490
16 -1.8134973031632078,-1.8236935502218616,0.42152938669069556,0.20821694217659054,0.9476191305116258,1.55907482385
17 -0.5615714999886664,-0.39998401699297917,-0.6684776277527593,-0.3338242295629643,-1.692442055394486,0.06254265391
18 -1.0467189673799582,-1.1822539557135165,1.6507496452584502,-0.6999010099168458,0.14556155648949972,-0.76385935850
19 0.3822853802478761,0.3002470614545819,0.20304463332613276,0.7287384050087065,0.7984762638384989,-0.60084955102237
20 -0.9338622463854552,-1.0144979868432984,1.2757273235235418,-0.3842975038761559,-0.048207142718927926,-0.441725582
21 -1.4768729298289265,-1.4184700562105004,0.15447827711739212,-0.9415541433132549,0.9828938149985281,0.282690286747
22 1.6221198864466961,2.01781105790233672,2.1646864304369138,0.6803958576453675,-0.6769927296031596,0.378696343542745
23 -0.19921461908680244,-0.5949850631067227,1.8566778320235764,0.1452016933655653,1.343515218105186,-0.6882392779045
24 0.11661705774273287,0.23605574495878517,-0.16470021237907564,1.261119705844415,2.2764077458678424,-0.153722801989
25 -1.4902732009371807,-1.781645535379393,1.066231638607476,-0.24709501992387228,-0.5150175317177772,1.9544162018404
26 -0.5422915216830843,-0.6358541693310412,0.73725804579204,0.5994715488996422,-0.6017091536029028,1.550884155824682
27 0.16334433654233715,0.3641548861987816,1.466674483418552,-0.20756619184182348,1.389697884774844,0.294519824532077
28 -1.1954929538927102,-1.1520116768556392,-0.8867582029150923,-1.4984192477512952,1.5220338329531644,-0.09304811482
29 0.44845576897521305,0.18992404113869535,0.778280037987183,0.24226922046324903,-0.6828357722732381,-0.013234525382
30 -0.6028682109888396,-0.7443123166072556,1.1538250095427918,-0.4100749439494218,-0.5751599846562345,0.466930945952
31 -0.27019483991392823,-0.1366207283883512,-1.687336105580332,0.4844794579667596,-0.43001436418385486,-1.1252090380
32 -0.6040181547780695,-0.7485300122006551,-1.1069530747576273,-1.5086235213150553,-1.5444568553294353,0.3804546445
33 2.0384442677034915,1.9836906449626188,-1.9702180873405521,2.440630713463148,-1.0979119798114425,-1.75356805579034
34 0.029368492548752033,0.01806047168801806,-0.18531703305576952,-0.08343852399085256,-1.1875660974692444,0.25842076
35 -1.0950457573291255,-1.2035825449952473,0.8100390784097634,-1.0718611745042894,-0.9397074752333526,0.499699810146
36 0.9296226152108739,1.3177949125179256,1.601539077336229,0.03646529116383452,-0.1708751289856153,-0.60016905810313
37 -1.2707078420056779,-1.1064520322878688,0.16111059430054198,-1.970964001196291,0.11112616764243867,1.401472849463
38 0.2688076737462981,-0.17836940810149313,1.72887306488212458,0.7071848553499354,1.4915457131858323,-1.3227030816480
```

Archivo Editar Selección Ver Ir a Ejecutar ... Practicas\_Ciencia\_Datos

EXPLORADOR ... ractica\_2.py Practica\_3.py Practica\_4.py Practica\_5.py Practica\_p.py Practica\_P

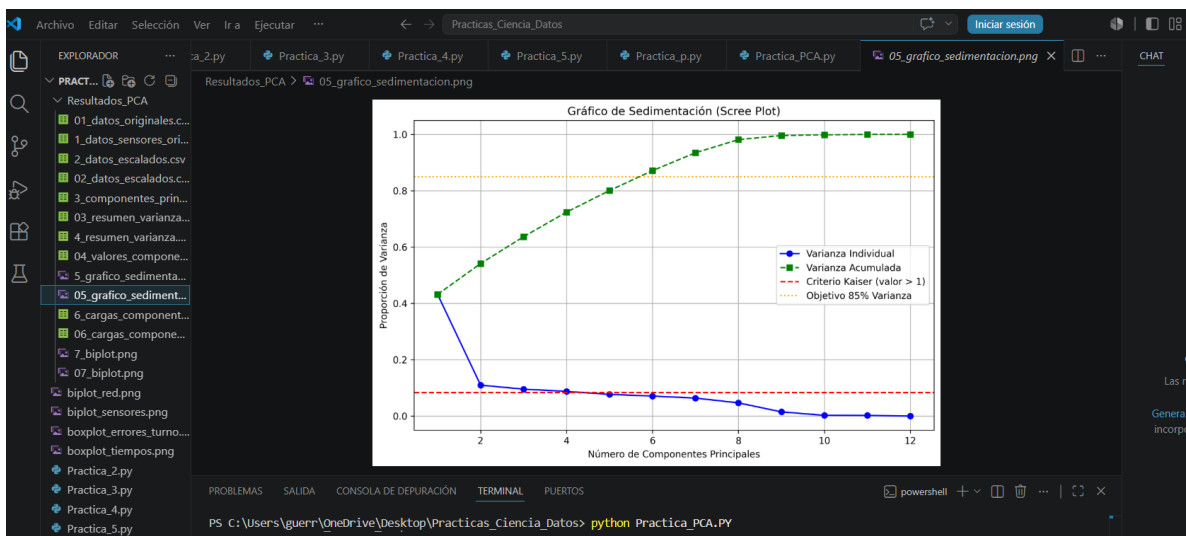
Resultados\_PCA > 03\_resumen\_varianza.csv

```
1 ,Varianza_Explicada_Individual,Varianza_Explicada_Acumulada
2 PC1,0.4317,0.4317
3 PC2,0.1095,0.5412
4 PC3,0.0954,0.6365
5 PC4,0.0873,0.7239
6 PC5,0.0766,0.8005
7 PC6,0.0707,0.8712
8 PC7,0.0635,0.9347
9 PC8,0.0466,0.9813
10 PC9,0.0146,0.9959
11 PC10,0.0023,0.9981
12 PC11,0.0019,1.0
13 PC12,0.0,1.0
14
```

```
Archivo  Editar  Selección  Ver  Ir a  Ejecutar  ...  <  >  Practicas_Ciencia_Datos  Iniciar sesión

EXPLORADOR  ...  tica_2.py  Practica_3.py  Practica_4.py  Practica_5.py  Practica_p.py  Practica_PCA.py  04_valores_componentes.csv  CHAT

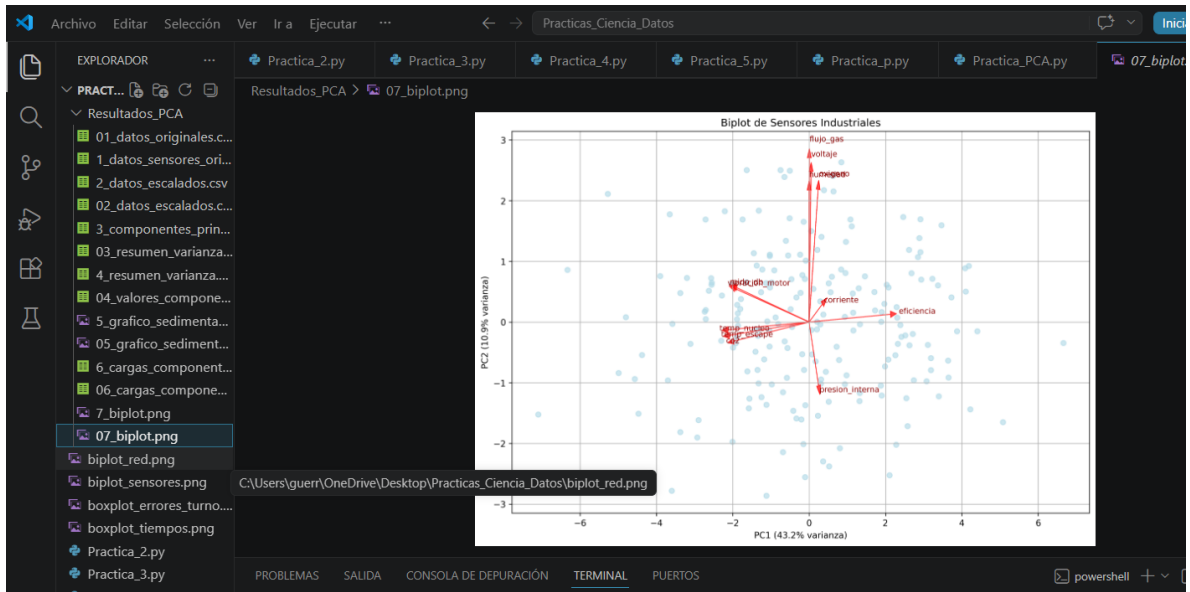
PRACTICAS_CIENCIA_DATOS  Resultados_PCA > 04_valores_componentes.csv
1 PC1,PC2,PC3,PC4,PC5,PC6,PC7,PC8,PC9,PC10,PC11,PC12
2 -1.6380892773154025,2.5040280376938764,1.8558175667819063,-0.10509434197294489,-0.08727758685144726,-0.38418212755
3 0.7811503066516885,-0.6231736103240346,1.8618130203721548,0.029351102047894585,0.8290449394958846,-0.202821765313
4 -1.9962788651631607,-0.4191671726723613,1.598805462415109,1.236502612924461,-0.07581067051559491,-0.4561902809283
5 -4.469944272440266,-1.5113641852672763,0.9627951964967887,0.9947640452405706,0.008812348904565788,-0.407794675644
6 0.3115848566275573,-0.9450214581820096,-0.6644531815150191,-0.5902830092911856,1.5810957666976202,-1.354323546654
7 -0.4672869121531068,0.7078103133595353,-1.62958590171784,-0.1021437435937033,-0.1951542273321808,1.53366707715
8 -3.374883200643768,0.4789962395246622,1.2032966906737461,-0.37090852816096426,0.636451797056197,0.413383004620537
9 -1.2498588468477094,-1.238911678754338,0.6093395102582843,-0.08338341913992021,1.1861999471205318,0.7100144064341
10 2.290832518201412,0.2454617376083702,2.0903650001756655,1.5183499104720852,0.6089034163225145,-0.9300863813921845
11 -2.7117880419959803,0.050990042293893305,-0.8357081084834872,-0.4081839615826241,-1.0830404988598115,0.78139997364
12 0.9758846899701359,-0.8758427856286446,-0.21894722442676043,0.8600575431681291,1.8038766085869968,-0.227641265129
13 0.9332011860609135,0.08851780797968456,0.2362333995766223,-1.2029422977278383,0.07113398008210994,-0.774089109946
14 -0.7497802920627897,2.504144905676958,0.6779272381132634,-0.407959933006183,1.1269902005866577,-0.34234263506262
15 5.075282032377306,-1.6505890860587318,0.2348510751038953,1.1426040290500734,-0.3245087415975285,-1.2048481992344
16 2.847160803217756,1.0697039718614185,-0.406578934898324,1.3849750600419293,1.3137206004002773,0.5403992905302901
17 0.804327385486424,-2.080071340995317,0.937379807838911,-0.6208243259657377,1.0702089175657956,-0.181696853078977
18 2.41328392059014,-0.24451074608436604,-1.5226418244180204,0.7720298222303998,-0.9124803611455863,0.73630476861374
19 -1.2615592319963076,0.49356673433288617,-1.6997482183795871,-0.14585748046549778,-0.3920172679622112,-0.215979792
20 1.875946542047237,-1.046423279380937,-1.895704395144583,0.540873796446715,0.3716792980199138,-0.5709191465161091
21 2.6158499531903843,1.1011760447243767,-0.0987764566390709,0.51293110788911,-0.4013520148450241,0.2028271729365297
22 -2.930858118382184,-1.9029897920442587,-0.08748384357663817,1.648455294493955,1.1295141122920638,1.1023397474755
23 0.2865165166552987,0.3430714807779665,-2.694446783366595,1.132717817671069,-0.46034768105592205,-0.7130723184566
```



```
Archivo  Editar  Selección  Ver  Ir a  Ejecutar  ...  <  >  Practicas_Ciencia_Datos

EXPLORADOR  ...  tica_2.py  Practica_3.py  Practica_4.py  Practica_5.py  Practica_p.py  Practica_PCA.py  06_cargas_com...

PRACT...  Resultados_PCA > 06_cargas_componentes.csv
1 ,PC1,PC2,PC3,PC4,PC5,PC6,PC7,PC8,PC9,PC10,PC11,PC12
2 temp_nucleo,-0.427,-0.026,0.049,-0.061,0.021,-0.005,0.028,-0.24,-0.256,-0.275,-0.334,0.707
3 temp_escape,-0.419,-0.044,0.091,-0.062,0.009,0.008,0.04,-0.262,-0.338,0.579,0.539,0.0
4 presion_interna,0.05,-0.21,-0.393,0.709,-0.055,0.37,0.33,-0.204,-0.078,-0.001,-0.008,-0.0
5 vibracion_motor,-0.388,0.11,-0.079,0.161,-0.029,0.002,0.01,0.526,-0.039,-0.508,0.517,0.0
6 flujo_gas,0.001,0.542,-0.199,0.426,-0.006,-0.515,-0.393,-0.256,-0.021,0.018,0.008,0.0
7 oxigeno,0.047,0.437,0.176,0.045,0.811,0.281,0.192,-0.015,-0.008,-0.003,0.007,0.0
8 co2,-0.395,-0.063,0.056,0.025,0.031,0.022,0.032,-0.299,0.859,-0.008,0.08,0.0
9 humedad,0.0,0.435,-0.469,-0.386,-0.217,-0.116,0.613,-0.061,0.04,0.013,0.011,0.0
10 voltaje,0.01,0.496,0.285,0.013,-0.499,0.615,-0.195,-0.078,0.006,-0.007,-0.012,0.0
11 corriente,0.07,0.06,0.666,0.313,-0.2,-0.355,0.531,0.031,-0.006,-0.015,-0.022,-0.0
12 ruido_db,-0.379,0.113,-0.087,0.173,-0.017,0.01,0.007,0.576,0.083,0.504,-0.461,-0.0
13 eficiencia,0.427,0.026,-0.049,0.061,-0.021,0.005,-0.028,0.24,0.256,0.275,0.334,0.707
14
```



## Práctica 5: Reducción de Dimensiones en Tráfico de Red (Cyber-Systems)

### Unidad 2: Tratamiento de Datos – TecNM

#### Código:

```
# Importar librerías necesarias

import numpy as np

import pandas as pd

from sklearn.decomposition import PCA

from sklearn.preprocessing import StandardScaler

import matplotlib.pyplot as plt


# -----

# Paso 1: Generación y Exploración de Datos

# -----

np.random.seed(123) # Equivalente a set.seed() en R

n = 300


# Crear DataFrame con las métricas simuladas

datos_red = pd.DataFrame({

    'duracion_ms': np.random.normal(50, 10, n),

    'paquetes_enviados': np.random.normal(100, 20, n),

    'bytes_enviados': np.nan, # Se calculará después

    'errores_checksum': np.random.poisson(2, n),
```

```

'reintentos_tcp': np.nan,

'latencia_avg': np.random.normal(15, 5, n),

'jitter': np.random.normal(2, 0.5, n),

'carga_cpu_router': np.nan,

'uso_memoria_sw': np.random.normal(40, 10, n),

'peticiones_http': np.random.normal(200, 50, n)

})

```

# Crear dependencias (redundancia) igual que en R

```

datos_red['bytes_enviados'] = datos_red['paquetes_enviados'] * 1500 +
np.random.normal(0, 500, n)

datos_red['reintentos_tcp'] = datos_red['errores_checksum'] * 1.5 +
np.random.normal(0, 0.5, n)

datos_red['carga_cpu_router'] = (datos_red['paquetes_enviados'] * 0.4) +
(datos_red['latencia_avg'] * 0.2)

```

# Mostrar las primeras filas

```

print("Primeras filas de los datos:")

print(datos_red.head(), "\n")

```

# -----

# Paso 2: Estandarización (Scaling)

# -----

# El PCA es sensible a las escalas, estandarizamos a media 0 y desviación estándar 1

```

escalador = StandardScaler()

datos_escalados = escalador.fit_transform(datos_red)

# Aplicar PCA (equivalente a prcomp con center=TRUE y scale.=TRUE)
pca = PCA(n_components=None) # Calcula todos los componentes
pca_resultados = pca.fit(datos_escalados)

# -----

# Paso 3: Análisis de Varianza

# -----

# Varianza explicada por cada componente
varianza_explicada = pca.explained_variance_ratio_
varianza_acumulada = np.cumsum(varianza_explicada)

print("=== Resumen de Varianza Explicada ===")
for i, (var, acum) in enumerate(zip(varianza_explicada, varianza_acumulada), 1):
    print(f'Componente {i}: Varianza = {var:.3f} | Acumulada = {acum:.3f}')

# Gráfica de codo (Scree Plot)

plt.figure(figsize=(8, 5))

plt.plot(range(1, len(varianza_explicada)+1), varianza_explicada, marker='o',
linestyle='-')

plt.title('Scree Plot: Tráfico de Red')

plt.xlabel('Número de Componente Principal')

```

```
plt.ylabel('Proporción de Varianza Explicada')
```

```
plt.grid(True)
```

```
plt.show()
```

```
# -----
```

```
# Paso 4: Interpretación de Cargas (Loadings)
```

```
# -----
```

```
# Obtener los pesos (cargas) de cada variable en los componentes
```

```
cargas = pd.DataFrame(
```

```
    pca.components_.T,
```

```
    index=datos_red.columns,
```

```
    columns=[f'PC{i+1}' for i in range(pca.n_components_)])
```

```
)
```

```
print("\n=== Cargas de las variables (PC1 y PC2) ===")
```

```
print(round(cargas[['PC1', 'PC2']], 3))
```

```
# Interpretación:
```

```
# - PC1: valores altos en paquetes_enviados, bytes_enviados, carga_cpu_router -  
> Volumen de Tráfico
```

```
# - PC2: valores altos en latencia_avg, jitter -> Calidad/Estabilidad de Conexión
```

```
# -----
```

```
# Paso 5: Visualización (Biplot)
```



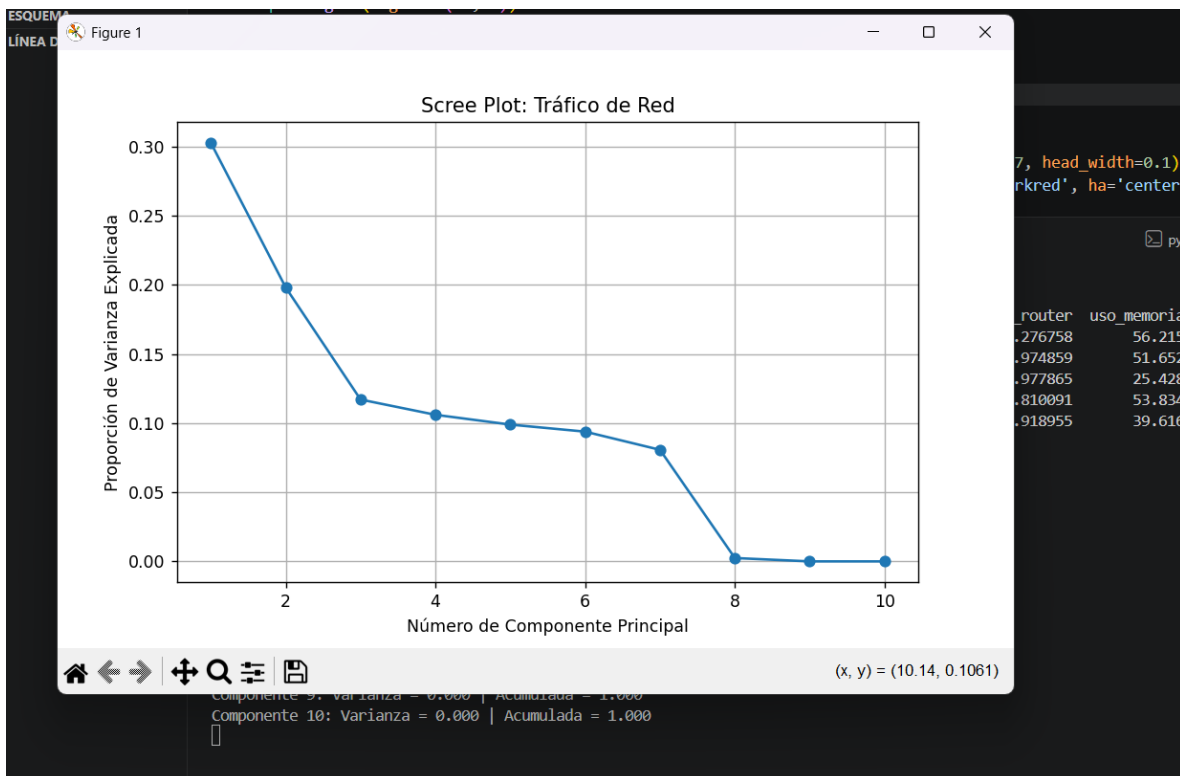
```
# -----  
  
# Proyectar los datos en los primeros 2 componentes  
proyeccion = pca.transform(datos_escalados)  
  
plt.figure(figsize=(10, 8))  
  
# Puntos de las observaciones  
plt.scatter(proyeccion[:, 0], proyeccion[:, 1], alpha=0.6, s=15)  
  
# Flechas de las variables originales  
escala_flechas = 3 # Ajusta para ver mejor las etiquetas  
for var, (x, y) in zip(datos_red.columns, cargas[['PC1', 'PC2']].values):  
    plt.arrow(0, 0, x * escala_flechas, y * escala_flechas, color='red', alpha=0.7,  
             head_width=0.1)  
    plt.text(x * escala_flechas * 1.15, y * escala_flechas * 1.15, var, color='darkred',  
            ha='center', va='center')  
  
plt.title('Mapa de Estado de Red (Biplot)')  
plt.xlabel(f'PC1 ({varianza_explicada[0]:.1%} varianza)')  
plt.ylabel(f'PC2 ({varianza_explicada[1]:.1%} varianza)')  
plt.axhline(0, color='gray', linestyle='--', alpha=0.5)  
plt.axvline(0, color='gray', linestyle='--', alpha=0.5)  
plt.grid(True)  
plt.show()
```

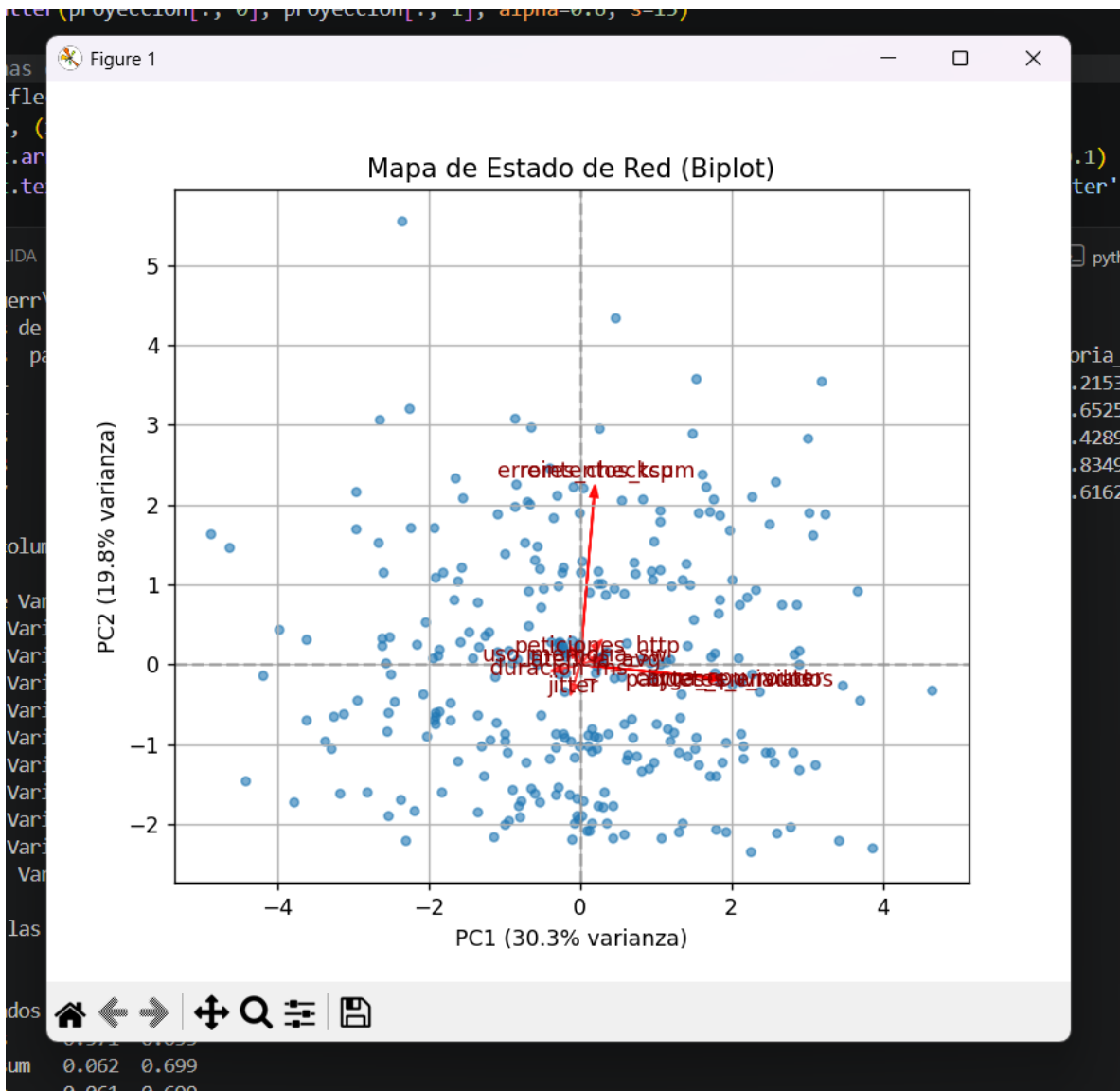
## Resultados:

```
PS C:\Users\guerr\OneDrive\Desktop\Practicas_Ciencia_Datos> python Practica5_Red_Red_Net.py
Primeras filas de los datos:
  duracion_ms  paquetes_enviados  bytes_enviados  errores_checksum  ...  jitter  carga_cpu_router  uso_memoria_sw  peticiones_http
0    39.143694      115.301097    173970.371351      2    ...  2.545339      49.276758      56.215306      169.596990
1    59.973454      83.420223    124270.498035      4    ...  2.345872      35.974859      51.652583      131.539070
2    52.829785      86.816974    130992.878118      4    ...  1.391032      37.977865      25.428999      234.862122
3    34.937053     112.222471    167584.966401      2    ...  2.642796      46.810091      53.834913      159.001548
4    44.213997      97.119733    146291.186256      2    ...  1.454608      40.918955      39.616217      257.375376

[5 rows x 10 columns]

=== Resumen de Varianza Explicada ===
Componente 1: Varianza = 0.303 | Acumulada = 0.303
Componente 2: Varianza = 0.198 | Acumulada = 0.501
Componente 3: Varianza = 0.117 | Acumulada = 0.618
Componente 4: Varianza = 0.106 | Acumulada = 0.724
Componente 5: Varianza = 0.099 | Acumulada = 0.823
Componente 6: Varianza = 0.094 | Acumulada = 0.917
Componente 7: Varianza = 0.081 | Acumulada = 0.998
Componente 8: Varianza = 0.002 | Acumulada = 1.000
Componente 9: Varianza = 0.000 | Acumulada = 1.000
Componente 10: Varianza = 0.000 | Acumulada = 1.000
[]
```





[5 rows x 10 columns]

=== Resumen de Varianza Explicada ===

|                |                  |  |                   |
|----------------|------------------|--|-------------------|
| Componente 1:  | Varianza = 0.303 |  | Acumulada = 0.303 |
| Componente 2:  | Varianza = 0.198 |  | Acumulada = 0.501 |
| Componente 3:  | Varianza = 0.117 |  | Acumulada = 0.618 |
| Componente 4:  | Varianza = 0.106 |  | Acumulada = 0.724 |
| Componente 5:  | Varianza = 0.099 |  | Acumulada = 0.823 |
| Componente 6:  | Varianza = 0.094 |  | Acumulada = 0.917 |
| Componente 7:  | Varianza = 0.081 |  | Acumulada = 0.998 |
| Componente 8:  | Varianza = 0.002 |  | Acumulada = 1.000 |
| Componente 9:  | Varianza = 0.000 |  | Acumulada = 1.000 |
| Componente 10: | Varianza = 0.000 |  | Acumulada = 1.000 |

=== Cargas de las variables (PC1 y PC2) ===

|                   | PC1    | PC2    |
|-------------------|--------|--------|
| duracion_ms       | -0.081 | -0.017 |
| paquetes_enviados | 0.571  | -0.056 |
| bytes_enviados    | 0.571  | -0.055 |
| errores_checksum  | 0.062  | 0.699  |
| reintentos_tcp    | 0.061  | 0.699  |
| latencia_avg      | 0.052  | 0.015  |
| jitter            | -0.027 | -0.081 |
| carga_cpu_router  | 0.572  | -0.054 |
| uso_memoria_sw    | -0.019 | 0.034  |
| peticiones_http   | 0.064  | 0.067  |

**title: "Práctica 6: Optimización de Sensores en Smart Cities"**  
**subtitle: "Unidad 2: Tratamiento de Datos - TecNM" author: "Guía Paso a Paso con Interpretación" output: html\_document: toc: true**  
**theme: united**

**Código:**

```
# Importar las librerías necesarias

import numpy as np

import pandas as pd

from sklearn.decomposition import PCA

from sklearn.preprocessing import StandardScaler

import matplotlib.pyplot as plt


# =====

# Paso 1: Generación de Datos Urbanos

# =====

np.random.seed(456) # Equivalente a set.seed(456) en R

n = 400


# Crear DataFrame con las mediciones de los sensores

datos_urbanos = pd.DataFrame({

    'temp_ambiente': np.random.normal(28, 4, n),

    'humedad_rel': np.random.normal(60, 10, n),

    'co2_ppm': np.nan,          # Se calculará después

    'no2_ppb': np.nan,         # Se calculará después
```

```

'particulas_pm25': np.nan,    # Se calculará después
'nivel_ruido_db': np.nan,    # Se calculará después
'densidad_vehiculos': np.random.normal(120, 30, n),
'velocidad_viento': np.random.normal(15, 5, n),
'radiacion_solar': np.random.normal(800, 100, n),
'conteo_peatones': np.random.normal(50, 15, n)
})

# Generar correlaciones y redundancia (lógica de ciudad)

datos_urbanos['co2_ppm'] = (datos_urbanos['densidad_vehiculos'] * 3.5) +
np.random.normal(300, 50, n)

datos_urbanos['no2_ppb'] = (datos_urbanos['co2_ppm'] * 0.1) +
np.random.normal(5, 2, n)

datos_urbanos['particulas_pm25'] = (datos_urbanos['co2_ppm'] * 0.05) +
np.random.normal(10, 3, n)

datos_urbanos['nivel_ruido_db'] = (datos_urbanos['densidad_vehiculos'] * 0.2) + 50
+ np.random.normal(0, 5, n)

# Mostrar las primeras filas para ver la estructura

print("=== Primeras filas de los datos de sensores ===")

print(datos_urbanos.head(), "\n")

# =====

# Paso 2: Estandarización de las Métricas

# =====

```

```
# Escalamos para que todas las variables tengan media 0 y desviación estándar 1
```

```
escalador = StandardScaler()
```

```
datos_escalados = escalador.fit_transform(datos_urbanos)
```

```
# Aplicamos PCA (equivalente a prcomp con center=TRUE y scale.=TRUE)
```

```
pca_ciudad = PCA()
```

```
pca_ciudad.fit(datos_escalados)
```

```
# =====
```

```
# Paso 3: Análisis de Varianza Acumulada
```

```
# =====
```

```
varianza_explicada = pca_ciudad.explained_variance_ratio_
```

```
varianza_acumulada = np.cumsum(varianza_explicada)
```

```
print("=== Resumen de Varianza Explicada ===")
```

```
for i, (var, acum) in enumerate(zip(varianza_explicada, varianza_acumulada), 1):
```

```
    print(f'Componente {i}: Varianza individual = {var:.3f} | Varianza acumulada = {acum:.3f}')
```

```
# Gráfica Scree Plot
```

```
plt.figure(figsize=(8, 5))
```

```
plt.plot(range(1, len(varianza_explicada)+1), varianza_explicada, marker='o',  
linestyle='-')
```

```
plt.axhline(y=1/len(datos_urbanos.columns), color='red', linestyle='--',  
label='Umbral referencia')
```

```
plt.title("Scree Plot: Sensores Smart City")
plt.xlabel("Número de Componentes Principales")
plt.ylabel("Proporción de Varianza Explicada")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

```
# =====
```

```
# Paso 4: Interpretación de los Componentes (Loadings)
```

```
# =====
```

```
# Convertimos los pesos en un DataFrame para verlo claro
```

```
cargas = pd.DataFrame(
    pca_ciudad.components_.T,
    index=datos_urbanos.columns,
    columns=[f'PC{i+1}' for i in range(pca_ciudad.n_components_)]
)
```

```
print("\n=== Pesos de las variables en PC1 y PC2 ===")
```

```
print(round(cargas[['PC1', 'PC2']], 3))
```

```
# Interpretación esperada:
```

```
# - PC1: Valores altos en densidad_vehiculos, co2_ppm, no2_ppb,
particulas_pm25, nivel_ruido_db → Representa: Actividad e Impacto Vehicular
```



# - PC2: Valores altos en temp\_ambiente, humedad\_rel, radiacion\_solar →  
Representa: Factor Climático / Meteorológico

# =====

# Paso 5: El Biplot de Diagnóstico Urbano

# =====

# Proyectamos los datos sobre los dos primeros componentes

proyeccion = pca\_ciudad.transform(datos\_escalados)

plt.figure(figsize=(10, 8))

# Dibujamos los puntos (mediciones de sensores)

plt.scatter(proyeccion[:, 0], proyeccion[:, 1], alpha=0.6, s=15)

# Dibujamos las flechas de cada variable original

escala\_flechas = 3 # Ajusta el tamaño de las flechas para ver bien las etiquetas

for variable, (x, y) in zip(datos\_urbanos.columns, cargas[['PC1', 'PC2']].values):

plt.arrow(0, 0, x \* escala\_flechas, y \* escala\_flechas,

color='red', alpha=0.7, head\_width=0.1)

plt.text(x \* escala\_flechas \* 1.15, y \* escala\_flechas \* 1.15,

variable, color='darkred', ha='center', va='center')

# Detalles del gráfico

plt.title("Mapa Biplot de Estado Urbano")

plt.xlabel(f"PC1: Actividad Vehicular ({varianza\_explicada[0]:.1%} varianza)")

```
plt.ylabel(f"PC2: Factor Climático ({varianza_explicada[1]:.1%} varianza)")
```

```
plt.axhline(0, color='gray', linestyle='--', alpha=0.5)
```

```
plt.axvline(0, color='gray', linestyle='--', alpha=0.5)
```

```
plt.grid(True, alpha=0.3)
```

```
plt.show()
```

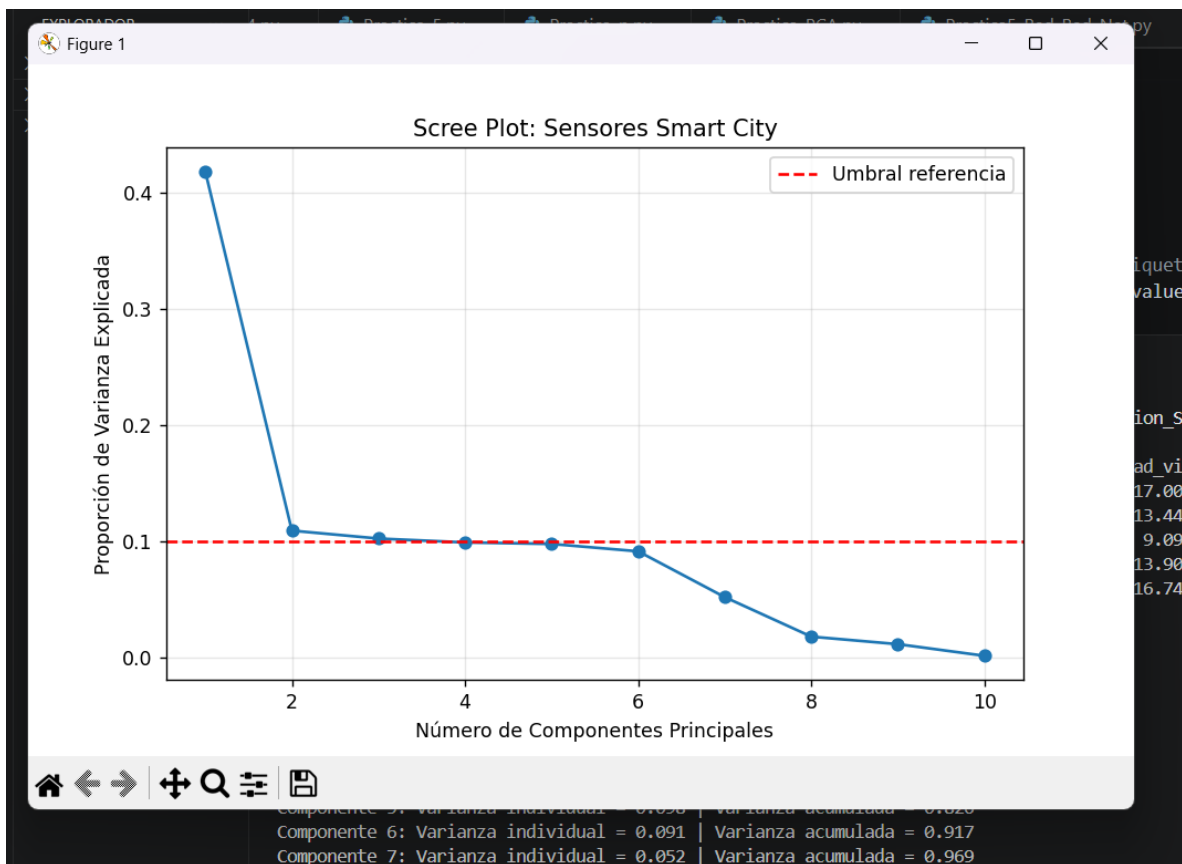
## Resultados:

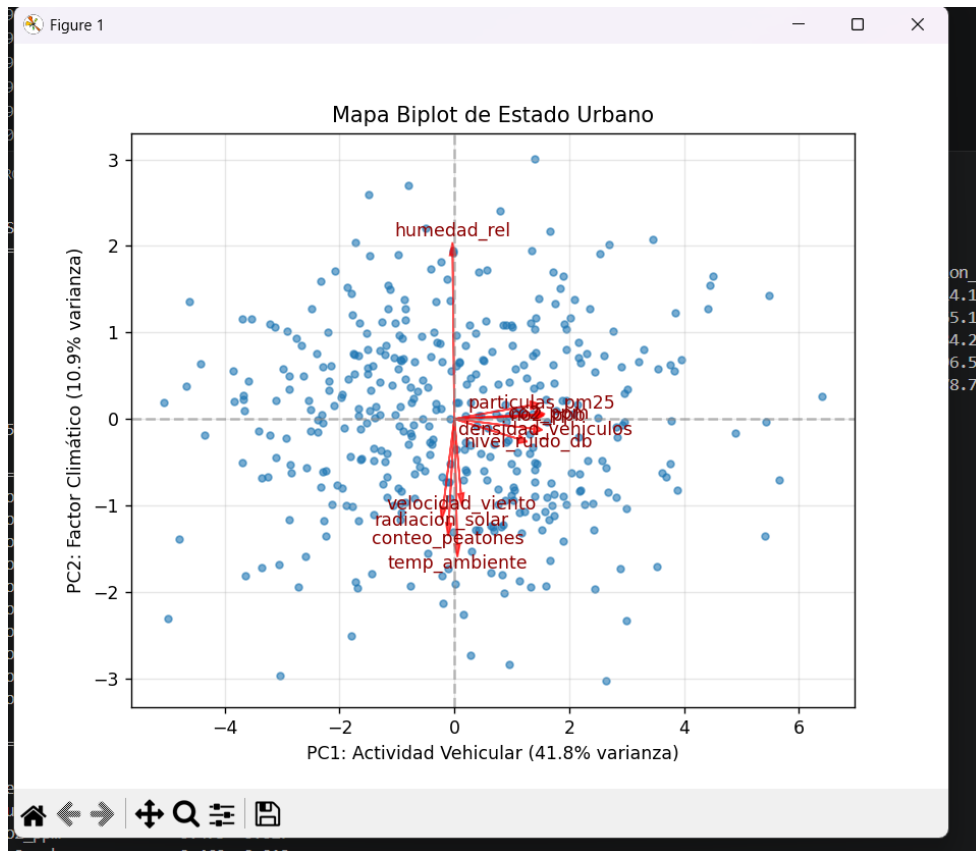
```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS
python + - [ ] [ ] ...

PS C:\Users\guerr\OneDrive\Desktop\Practicas_Ciencia_Datos> python Practica6_Optimizacion_Sensores.py
=== Primeras filas de los datos de sensores ===
temp_ambiente  humedad_rel  co2_ppm  no2_ppb  ...  densidad_vehiculos  velocidad_viento  radiacion_solar  conteo_peatones
0      25.327486    45.261746   781.712434  81.631038  ...      143.849039      17.005535      914.157550      57.722465
1      26.007162    67.870763   666.192972  71.192976  ...      105.765522      13.442578      965.173743      38.786281
2      30.474303    63.935109   753.040695  78.155747  ...      118.528576       9.093427      834.292432      52.585696
3      30.274769    50.079965   579.609581  62.981791  ...       76.401813      13.901557      796.567047      42.961262
4      33.402038    68.525178   667.118803  71.646128  ...       87.796920      16.746030      678.768816      19.779909

[5 rows x 10 columns]

=== Resumen de Varianza Explicada ===
Componente 1: Varianza individual = 0.418 | Varianza acumulada = 0.418
Componente 2: Varianza individual = 0.109 | Varianza acumulada = 0.527
Componente 3: Varianza individual = 0.102 | Varianza acumulada = 0.629
Componente 4: Varianza individual = 0.099 | Varianza acumulada = 0.728
Componente 5: Varianza individual = 0.098 | Varianza acumulada = 0.826
Componente 6: Varianza individual = 0.091 | Varianza acumulada = 0.917
Componente 7: Varianza individual = 0.052 | Varianza acumulada = 0.969
Componente 8: Varianza individual = 0.018 | Varianza acumulada = 0.987
Componente 9: Varianza individual = 0.011 | Varianza acumulada = 0.999
Componente 10: Varianza individual = 0.001 | Varianza acumulada = 1.000
[]
```





```

PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS
python + - [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]

PS C:\Users\guerr\OneDrive\Desktop\Practicas_Ciencia_Datos> python Practica6_Optimizacion_Sensores.py
3    30.274769    50.079965    579.609581    62.981791    ...    76.401813    13.901557    796.567047    42.961262
4    33.402038    68.525178    667.118803    71.646128    ...    87.796920    16.746030    678.768816    19.779909

[5 rows x 10 columns]

=== Resumen de Varianza Explicada ===
Componente 1: Varianza individual = 0.418 | Varianza acumulada = 0.418
Componente 2: Varianza individual = 0.109 | Varianza acumulada = 0.527
Componente 3: Varianza individual = 0.102 | Varianza acumulada = 0.629
Componente 4: Varianza individual = 0.099 | Varianza acumulada = 0.728
Componente 5: Varianza individual = 0.098 | Varianza acumulada = 0.826
Componente 6: Varianza individual = 0.091 | Varianza acumulada = 0.917
Componente 7: Varianza individual = 0.052 | Varianza acumulada = 0.969
Componente 8: Varianza individual = 0.018 | Varianza acumulada = 0.987
Componente 9: Varianza individual = 0.011 | Varianza acumulada = 0.999
Componente 10: Varianza individual = 0.001 | Varianza acumulada = 1.000

=== Pesos de las variables en PC1 y PC2 ===
      PC1    PC2
temp_ambiente    0.016 -0.482
humedad_rel     -0.010  0.630
co2_ppm          0.475  0.017
no2_ppb          0.469  0.012
partículas_pm25  0.441  0.052
nivel_ruido_db   0.374 -0.078
densidad_vehiculos 0.461 -0.038
velocidad_viento  0.038 -0.287
radiacion_solar  -0.065 -0.339
conteo_peatones  -0.032 -0.402

```

## Práctica 7: Implementación de Modelos de Machine Learning

### Unidad 3: Modelado - Ciencia de Datos (TecNM)

#### Código:

```
# Importar librerías necesarias

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.neighbors import KNeighborsClassifier

from sklearn.preprocessing import StandardScaler

from sklearn.cluster import KMeans

from sklearn.metrics import accuracy_score, mean_squared_error

import math


# =====

# Paso 1: Preparación del Dataset

# =====

np.random.seed(789) # Equivalente a set.seed(789) en R

n = 500


# Crear variables predictoras

df_empleados = pd.DataFrame({
```

```

'experiencia': np.random.normal(5, 2, n),
'certificaciones': np.random.poisson(3, n),
'habilidades_sociales': np.random.uniform(1, 10, n),
'remoto': np.random.choice([0, 1], size=n, replace=True)
})

# Variable objetivo continua: Salario
df_empleados['salario'] = 25000 + (df_empleados['experiencia'] * 5000) + \
    (df_empleados['certificaciones'] * 2000) + np.random.normal(0,
3000, n)

# Variable objetivo binaria: Retención (1=Se queda, 0=Se va)

# Función logística para probabilidad
def logistica(x):
    return 1 / (1 + np.exp(-x))

prob = logistica(-2 + 0.0001 * df_empleados['salario'] + 0.2 *
df_empleados['habilidades_sociales'])

df_empleados['retencion'] = np.where(np.random.uniform(size=n) < prob, 1, 0)

df_empleados['retencion'] = df_empleados['retencion'].astype('category')

print("=== Primeras filas del conjunto de datos ===")

print(df_empleados.head(), "\n")

```

```

# =====

# Paso 2: Regresión Lineal (Predecir Salario)

# =====

# División 80% entrenamiento, 20% prueba

X_reg = df_empleados[['experiencia', 'certificaciones']]
y_reg = df_empleados['salario']


X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
    X_reg, y_reg, test_size=0.2, random_state=123
)


# Entrenamiento del modelo

modelo_regresion = LinearRegression()
modelo_regresion.fit(X_train_reg, y_train_reg)


# Resultados y coeficientes

print("=== Modelo de Regresión Lineal ===")
print(f"Intercepto: {modelo_regresion.intercept_:.2f}")
print(f"Coeficientes:")
for nombre, coef in zip(X_reg.columns, modelo_regresion.coef_):
    print(f" - {nombre}: {coef:.2f}")


# Evaluación

```

```
y_pred_reg = modelo_regresion.predict(X_test_reg)
mse = mean_squared_error(y_test_reg, y_pred_reg)
print(f"Error Cuadrático Medio (MSE): {mse:.2f}")
print(f"Raíz del Error Cuadrático Medio (RMSE): {math.sqrt(mse):.2f}\n")
```

```
# =====
```

```
# Paso 3: Clasificación KNN (Predecir Retención)
```

```
# =====
```

```
# Variables para clasificación
```

```
X_clf = df_empleados[['experiencia', 'habilidades_sociales', 'salario']]
```

```
y_clf = df_empleados['retencion']
```

```
X_train_clf, X_test_clf, y_train_clf, y_test_clf = train_test_split(
```

```
    X_clf, y_clf, test_size=0.2, random_state=123
```

```
)
```

```
# Escalamiento (obligatorio para KNN)
```

```
escalador = StandardScaler()
```

```
X_train_esc = escalador.fit_transform(X_train_clf)
```

```
X_test_esc = escalador.transform(X_test_clf)
```

```
# Probar diferentes valores de K (equivalente a tuneLength=5)
```

```
valores_k = [1, 3, 5, 7, 9]
```



```
precisiones = []
```

```
print("=== Desempeño KNN según valor de K ===")
```

```
for k in valores_k:
```

```
    modelo_knn = KNeighborsClassifier(n_neighbors=k)
```

```
    modelo_knn.fit(X_train_esc, y_train_clf)
```

```
    pred = modelo_knn.predict(X_test_esc)
```

```
    acc = accuracy_score(y_test_clf, pred)
```

```
    precisiones.append(acc)
```

```
    print(f"K = {k} | Precisión: {acc:.4f}")
```

```
# Seleccionar y entrenar el mejor modelo
```

```
mejor_k = valores_k[precisiones.index(max(precisiones))]
```

```
modelo_knn_final = KNeighborsClassifier(n_neighbors=mejor_k)
```

```
modelo_knn_final.fit(X_train_esc, y_train_clf)
```

```
print(f"\nMejor modelo: K = {mejor_k} con precisión de {max(precisiones):.4f}\n")
```

```
# =====
```

```
# Paso 4: Clustering No Supervisado (K-Means)
```

```
# =====
```

```
# Seleccionar y escalar datos
```

```
datos_clust = df_empleados[['experiencia', 'salario']].copy()
```

```
datos_clust_esc = StandardScaler().fit_transform(datos_clust)
```

```
# Entrenar K-Means con 3 grupos

kmeans = KMeans(n_clusters=3, random_state=456, n_init='auto')

df_empleados['cluster'] = kmeans.fit_predict(datos_clust_esc)

df_empleados['cluster'] = df_empleados['cluster'].astype('category')


# Visualización de los grupos

plt.figure(figsize=(10, 6))

colores = ['#1f77b4', '#ff7f0e', '#2ca02c']

etiquetas = ['Junior', 'Medio', 'Senior'] # Interpretación esperada


for i in range(3):

    datos_grupo = df_empleados[df_empleados['cluster'] == i]

    plt.scatter(

        datos_grupo['experiencia'],

        datos_grupo['salario'],

        color=colores[i],

        label=etiquetas[i],

        alpha=0.7

    )


plt.title("Segmentación de Empleados por Perfil (K-Means)")

plt.xlabel("Experiencia (años)")
```

```
plt.ylabel("Salario ($)")
```

```
plt.legend()
```

```
plt.grid(True, alpha=0.3)
```

```
plt.show()
```

## Resultados:

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS

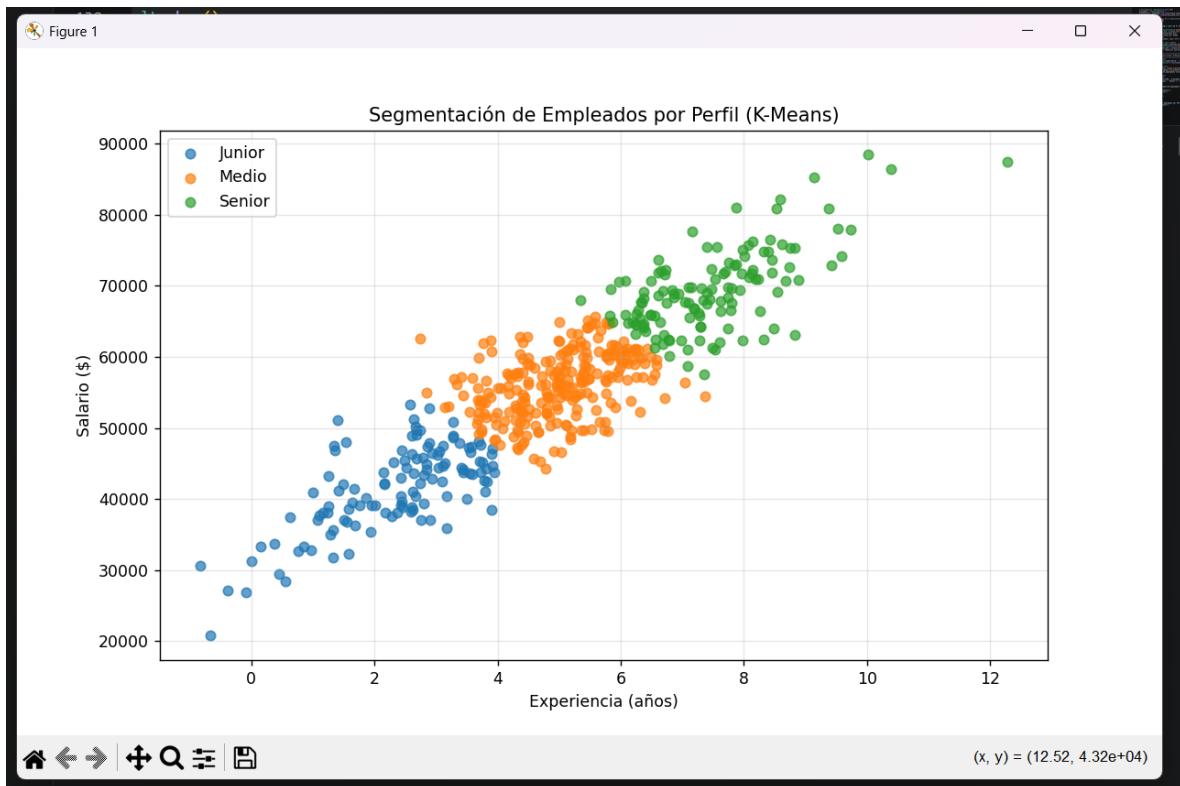
PS C:\Users\guerr\OneDrive\Desktop\Practicas_Ciencia_Datos> python Practica7_Modelos_ML.py
=== Primeras filas del conjunto de datos ===
  experiencia  certificaciones  habilidades_sociales  remoto  salario  retencion
0      2.783777             4             2.959476      0  45873.792275      1
1      3.548563             1             6.151329      0  43600.822391      1
2      6.045609             3             3.615433      0  62247.070746      1
3      7.468884             5             3.961266      1  72326.083333      1
4      5.193792             2             4.174604      0  49629.856459      1

=== Modelo de Regresión Lineal ===
Intercepto: 25575.49
Coeficientes:
- experiencia: 4939.48
- certificaciones: 2020.28
Error Cuadrático Medio (MSE): 10855214.38
Raíz del Error Cuadrático Medio (RMSE): 3294.73

=== Desempeño KNN según valor de K ===
K = 1 | Precisión: 0.9600
K = 3 | Precisión: 0.9700
K = 5 | Precisión: 0.9700
K = 7 | Precisión: 0.9700
K = 9 | Precisión: 0.9700

Mejor modelo: K = 3 con precisión de 0.9700

[]
```



Mejor modelo: K = 3 con precisión de 0.9700

PS C:\Users\guerr\OneDrive\Desktop\Practicas\_Ciencia\_Datos>

## Práctica 8: Clasificación de Intrusiones con KNN y Logística

### Unidad 3: Modelado – TecNM

#### Código:

```
# Importar librerías necesarias

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.linear_model import LogisticRegression

from sklearn.neighbors import KNeighborsClassifier

from sklearn.preprocessing import StandardScaler

from sklearn.model_selection import cross_val_score

from sklearn.cluster import KMeans


# =====

# Paso 1: Simulación de Tráfico de Red

# =====

np.random.seed(444)

n = 400


# Generar variables de entrada

df_red = pd.DataFrame({

    'latencia': np.random.normal(20, 5, n),

    'intentos_fallidos': np.random.poisson(1, n),
```

```

        'tamano_paquete': np.random.normal(500, 100, n)

    })

# Regla de clasificación: Ataque (1) si intentos > 2 o latencia > 35, de lo contrario
Seguro (0)

df_red['es_ataque'] = np.where(

    (df_red['intentos_fallidos'] > 2) | (df_red['latencia'] > 35),

    1, 0

)

print("=== Primeras filas del conjunto de datos ===")

print(df_red.head(), "\n")

# Separar características y variable objetivo

X = df_red[['latencia', 'intentos_fallidos', 'tamano_paquete']]

y = df_red['es_ataque']

# =====

# Paso 2: Modelo de Regresión Logística

# =====

modelo_log = LogisticRegression()

modelo_log.fit(X, y)

print("=== Resumen Regresión Logística ===")

```

```

print(f"Intercepto: {modelo_log.intercept_[0]:.4f}")

print("Coeficientes de las variables:")

for nombre, coef in zip(X.columns, modelo_log.coef_[0]):

    print(f" - {nombre}: {coef:.4f}")

print("\nInterpretación: Valores de coeficientes altos indican mayor peso para
detectar ataques\n")


# =====

# Paso 3: Clasificación con KNN (con Validación Cruzada)

# =====

# Escalamiento obligatorio para KNN

escalador = StandardScaler()

X_escalado = escalador.fit_transform(X)


# Probar valores de k de 1 a 10

k_valores = list(range(1, 11))

precisiones = []


# Validación cruzada de 5 pliegues

for k in k_valores:

    modelo_knn = KNeighborsClassifier(n_neighbors=k)

    scores = cross_val_score(modelo_knn, X_escalado, y, cv=5, scoring='accuracy')

    precisiones.append(scores.mean())

```

```

# Gráfico de precisión según valor de K

plt.figure(figsize=(8, 5))

plt.plot(k_valores, precisiones, marker='o', linestyle='-', color='green')

plt.title('Precisión del modelo KNN según número de vecinos')

plt.xlabel('Número de vecinos (k)')

plt.ylabel('Precisión media (Validación 5-fold)')

plt.xticks(k_valores)

plt.grid(True, alpha=0.3)

plt.show()


# Encontrar el mejor K

mejor_k = k_valores[precisiones.index(max(precisiones))]

print(f"=== Mejor valor de K: {mejor_k} con precisión de {max(precisiones):.4f} ===\n")


# =====

# Paso 4: Análisis de Resultados (Clustering K-Means)

# =====

# Usamos solo las métricas, SIN la etiqueta de ataque

datos_clust = X_escalado.copy()


# Aplicar K-Means con 2 grupos

kmeans = KMeans(n_clusters=2, random_state=111)

df_red['cluster'] = kmeans.fit_predict(datos_clust)

```



# Tabla de comparación: Etiqueta real vs Cluster asignado

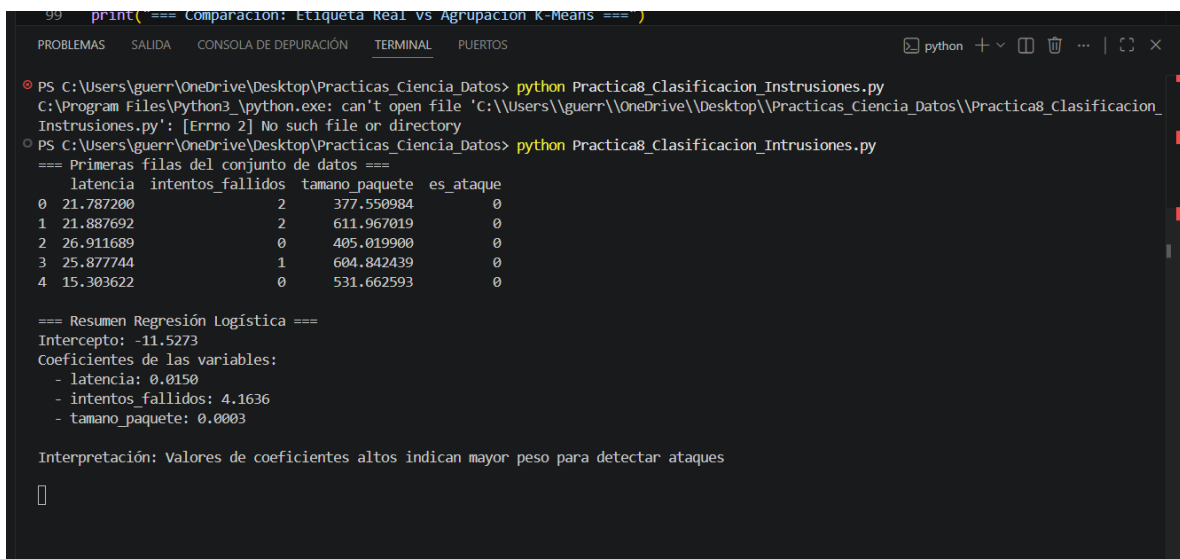
```
tabla_comparacion = pd.crosstab(  
  
    df_red['es_ataque'],  
  
    df_red['cluster'],  
  
    rownames=['Real: Es Ataque'],  
  
    colnames=['Cluster Asignado']  
  
)
```

```
print("=== Comparación: Etiqueta Real vs Agrupación K-Means ===")
```

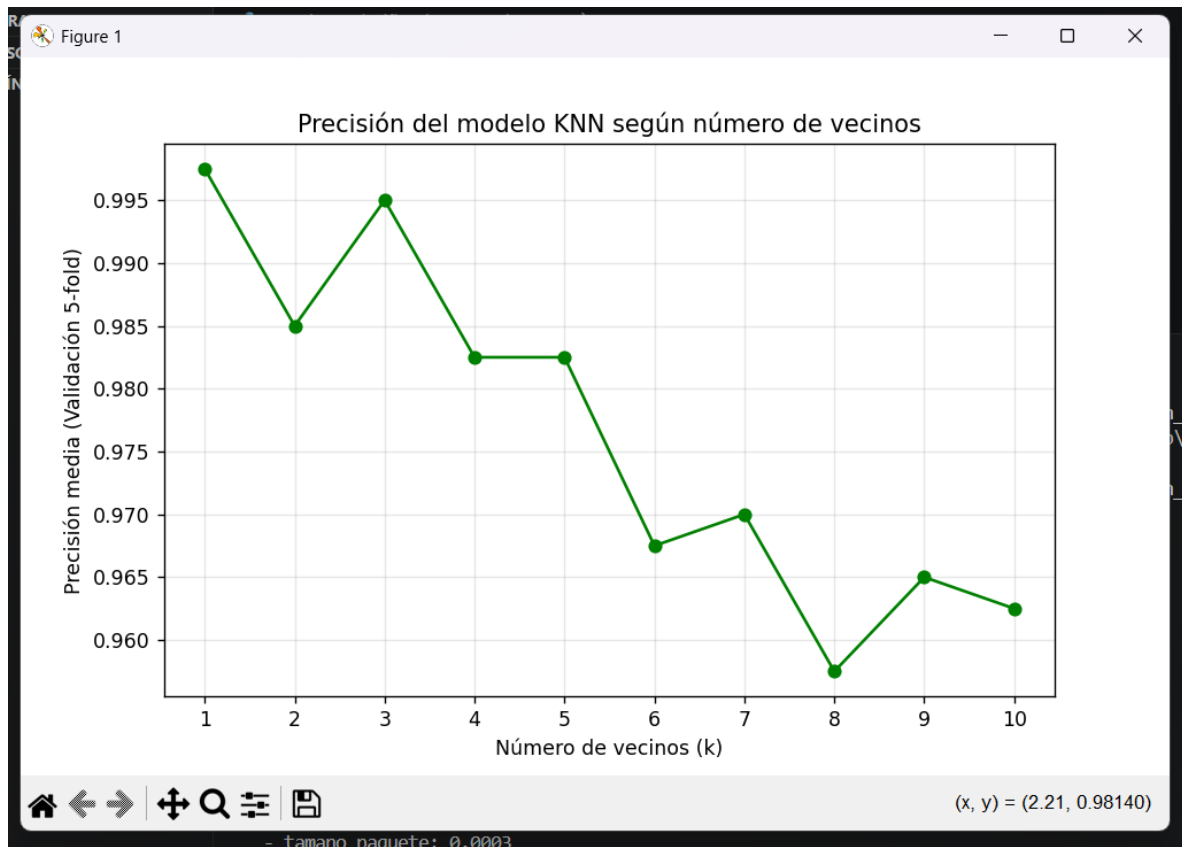
```
print(tabla_comparacion)
```

```
print("\nInterpretación: Si un cluster agrupa mayormente 1s y el otro 0s, el  
algoritmo detectó patrones sin supervisión.")
```

## Resultados:



```
99 print("=== Comparación: Etiqueta Real vs Agrupación K-Means ===")  
  
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS  
PS C:\Users\guerr\OneDrive\Desktop\Practicas_Ciencia_Datos> python Practica8_Clasificacion_Instrusiones.py  
C:\Program Files\Python3\python.exe: can't open file 'C:\\Users\\guerr\\OneDrive\\Desktop\\Practicas_Ciencia_Datos\\Practica8_Clasificacion_Instrusiones.py': [Error 2] No such file or directory  
PS C:\Users\guerr\OneDrive\Desktop\Practicas_Ciencia_Datos> python Practica8_Clasificacion_Intrusiones.py  
=== Primeras filas del conjunto de datos ===  
latencia  intentos_fallidos  tamaño_paquete  es_ataque  
0  21.787200                2          377.550984         0  
1  21.887692                2          611.967019         0  
2  26.911689                0          405.019900         0  
3  25.877744                1          604.842439         0  
4  15.303622                0          531.662593         0  
  
=== Resumen Regresión Logística ===  
Intercepto: -11.5273  
Coeficientes de las variables:  
- latencia: 0.0150  
- intentos_fallidos: 4.1636  
- tamaño_paquete: 0.0003  
  
Interpretación: Valores de coeficientes altos indican mayor peso para detectar ataques  
  
[]
```



=== Resumen Regresión Logística ===

Intercepto: -11.5273

Coefficientes de las variables:

- latencia: 0.0150
- intentos\_fallidos: 4.1636
- tamaño\_paquete: 0.0003

Interpretación: Valores de coeficientes altos indican mayor peso para detectar ataques

=== Mejor valor de K: 1 con precisión de 0.9975 ===

=== Comparación: Etiqueta Real vs Agrupación K-Means ===

Cluster Asignado    0    1

Real: Es Ataque

0                    218   151

1                    6     25

Interpretación: Si un cluster agrupa mayormente 1s y el otro 0s, el algoritmo detectó patrones sin supervisión.

PS C:\Users\guerr\OneDrive\Desktop\Practicas\_Ciencia\_Datos>